



Network configuration distributed production scheduling problem: A constraint programming approach

Ghazal Ziadlou, Saeed Emami^{*}, Ebrahim Asadi-Gangraj

Department of Industrial Engineering, Babol Noshirvani University of Technology, Babol 47148-71167, Iran

ARTICLE INFO

Keywords:

Distributed scheduling
Multi-factory scheduling
Network configuration
Constraint programming
Epsilon constraint method

ABSTRACT

Nowadays, many massive factories are forced to distribute their products in several manufacturing units. This issue has caused the emergence of a novel category of problems called distributed production scheduling, which is vital in today's growing world. In this paper, the distributed production scheduling problem by considering network configuration with two echelons is addressed. The first and second echelon factories have different job configurations and have a hybrid flow shop and a flexible job shop environment, respectively. For this problem, A bi-objective mixed integer linear programming (MILP) model is presented to minimize the maximum completion time of jobs and transportation costs between the selected factories in two echelons, respectively. Consequently, the epsilon constraint method is used to deal with this bi-objective model. In addition, since distributed scheduling problems are classified as NP-Hard problems, it is very challenging to solve them for large-sized instances. For this reason, a constraint programming model (CP) is also proposed. To evaluate the performance of the proposed MILP model and CP model, a total of 180 numerical instances are randomly generated in small, medium, and large sizes. The obtained results demonstrate the significant ability of the constraint programming approach in solving complex distributed scheduling problems even for large-sized instances with 30 jobs, 10 stages/operations for each job, 6 machines for each stage/operation, and 4 factories at each echelon in a reasonable time and proof that the CP model can outperform the MILP model in this problem.

1. Introduction

Sequencing and production scheduling represents a kind of decision-making process that plays a crucial role in the manufacturing and service industries. Such problems aim to allocate available resources to jobs over specific periods, to optimize one or more objective functions under different constraints. In recent decades, with increasing competition among various industries, having effective and efficient production planning and scheduling determines the survival of each company in the global arena (Pinedo, 2012). Every manufacturing and service industry seeks to produce products and provide services with the optimal and efficient use of present resources. It tries to do it in the shortest possible time to the customers committed to providing products or services; otherwise, it will deal with many unpleasant consequences.

Since the beginning of the last century, the production scheduling problem in the industry has received a great deal of attention, thanks to the efforts of Henry Gantt, one of the pioneers in this field. After that, it required several years for these problems related to production planning and scheduling to be earnestly studied and researched. The first

published paper on this subject was written in the early 1950s by Smith (1955), then in the 1960s, with the advent of dynamic programming and integer programming formulations for scheduling problems, this field witnessed significant progress. In the 1970s, after stating the complexity of scheduling problems and presenting the complexity hierarchy in a paper by Karp (1972), efforts continued to present and solve various problems in this area. From the 1980s to the present, with the advancement of technology and the use of computer software, the growth and development of these problems are absolutely remarkable (Pinedo, 2012). However, until the 1980s, production scheduling problems in the industry were always focused on production planning and scheduling for one factory. As mentioned earlier, nowadays, due to the increasing and more complex customer demands, many factories have been forced to produce their products in several manufacturing units.

Distributed manufacturing, also referred to as multi-factory manufacturing in the literature review, can obtain many advantages, such as increasing the quality of products that are produced and reducing production costs. Moreover, controlling and managing risk in

^{*} Corresponding author.

E-mail addresses: gh.ziadloo@stu.nit.ac.ir (G. Ziadlou), s_emami@nit.ac.ir (S. Emami), e.asadi@nit.ac.ir (E. Asadi-Gangraj).

<https://doi.org/10.1016/j.cie.2024.109916>

Received 15 March 2023; Received in revised form 17 December 2023; Accepted 17 January 2024

Available online 23 January 2024

0360-8352/© 2024 Elsevier Ltd. All rights reserved.

uncertain global market conditions and giving the ability for manufacturing plants to react in less time to market changes are the other benefits of this kind of production environment (Lohmer and Lasch, 2021); (Olhager and Feldmann, 2018); (Toptal and Sabuncuoglu, 2010).

According to Toptal & Sabuncuoglu (2010), distributed scheduling can be defined as follows: a scheduling problem divided into smaller problems (sub-problems), and each problem can be decided by local decision-makers who are also referred to as agents. In such cases, there may be conflicting objective functions but the ultimate goal is that through the participation of all agents, the whole system achieves a pleasing result. In general, in these problems, several jobs have to be distributed among several factories that have a particular shop configuration. In each problem, specific constraints and conditions can be considered, and ultimately, the goal is to determine the optimal schedule for the entire manufacturing network so that all limitations and constraints are observed. In other words, the ultimate aim of solving production scheduling problems in distributed environments, like single-factory problems, is to optimize one or more objective functions, considering the system. In fact, in the case of distributed production scheduling, the allocation of the factories to process the jobs is added to the single-factory production scheduling problem. So, generally, the three sub-problems of factory allocation, assignment of operations to machines, and sequencing on machines must be solved, simultaneously (P.-S. Wang et al., 2018).

In the real world, several industries such as the automobile industry are working in a distributed production environment to manufacture various types of necessary parts. Currently, more than 500 manufacturing companies are active in the production chain of the automobile industry in Iran. This includes many industries, such as metal industries, casting, textiles, chemicals, and other related sectors. Similarly, on average, a car consists of 6 to 14 thousand parts. It is naturally not possible to make all these parts in one factory, so it is necessary to distribute the production among different units. In some cases, all the processes needed to make a part can be done in one factory, while in others, cooperation between several manufacturing units is necessary to complete the production of a part. Finally, the assembly operation is conducted and completed. With these considerations, the necessity of high precision and attention to production planning and scheduling for this kind of industry is undeniable.

In addition, one of the most noteworthy applications of distributed scheduling models is their use in cloud manufacturing systems. Cloud manufacturing systems have been formed with the development of concepts such as globalization, integration, and emerging technologies such as cloud computing, Internet of Things (IoT), Cyber-Physical Systems (CPS), big data analysis, and Artificial Intelligence (AI). It is seeking to increase productivity and flexibility, reduce production costs, optimize resources allocated to jobs, and raise the quality of products produced in such systems. In this type of smart production system, the resources and different production capabilities of each factory in a system are considered production services that can be managed and operated in an intelligent and integrated way to enable their sharing and full circulation throughout the system. Therefore, the existence of integrated production scheduling in such complex systems is considered a vital tool to achieve these goals as best as possible (Jiang et al., 2022).

Hereupon, in this study, a mathematical model that tries to communicate between some distinct and separate factories through a single schedule provided by a central authority has been proposed. This effort can be considered a unique achievement in the direction of the related research carried out so far. Therefore, a bi-objective mixed integer linear programming (MILP) model is presented for the distributed production scheduling problem considering a two-echelon network configuration. It should be noted that the first and second echelon factories have a hybrid flow shop and flexible job shop environment, respectively. The objective functions of the presented model seek to minimize the maximum completion time of jobs and transportation costs

between echelons, respectively. As a consequence, to solve the proposed bi-objective model, the epsilon constraint method is utilized.

As it is clear, most of the production scheduling problems even in a single factory environment are complex and considered NP-Hard problems, and for this reason, the complexity of production scheduling problems in a distributed environment can be fully predicted and proven. Therefore, it can be concluded that the simplest production scheduling problem in the distributed environment is the case where there are two factories, so the minimum complexity of the distributed scheduling problems is at least the same as any other single-factory scheduling problem (Bargaoui et al., 2017); (Wu et al., 2017). This complexity increases several times by adding several limitations and assumptions, such as considering factories with different shop configurations, paying attention to time and transportation costs between factories, and so on.

For this reason, finding a good and powerful solution method to deal with these kinds of problems is undeniably crucial. CP is a powerful paradigm for solving combinatorial optimization problems that use a broad range of techniques such as artificial intelligence, operations research, graph theory, and so on to solve complex problems (Rossi et al., 2006). This method is recognized for its intense power in finding good-quality feasible solutions for high-dimensional combinatorial optimization problems in a reasonable time. To solve a problem, the CP method requires fewer variables and constraints compared to using mathematical modeling by any of the optimization software, which leads to the reduction of the solution time. The reason is the ability of the CP approach to define complex relationships between variables by global constraints and also the existence of effective constraint propagation algorithms in it (Meng et al., 2020).

The rest of this paper is organized as follows: Section 2 examines the literature on the distributed scheduling problem and eventually, our contribution to this field will be defined. Section 3 presents the MILP model for the considered problem. The CP model is introduced in Section 4. Section 5 describes the epsilon constraint method to solve the proposed bi-objective model. Section 6 defines a numerical example and analyzes the MILP and CP models. Finally, conclusions and suggestions for future works are presented in Section 7.

2. Literature review

The first paper on production scheduling in distributed environments (Williams, 1981) was published in 1981. In this study, seven heuristic algorithms for solving scheduling problems were discussed, and also an algorithm based on dynamic programming was proposed to combine production and distribution scheduling in a production network that included several factories. Each algorithm sought to determine an appropriate production or distribution schedule for the proposed model that could meet all existing demands by minimizing the average inventory maintenance and fixed processing costs. It should be noted that in the literature review related to this field, there are different classifications for problems related to distributed production environments according to the shop configuration of factories, the number of agents, and the structure of factories (Lohmer and Lasch, 2021).

Bonomi & Kumar (1990) presented an adaptive model for load balancing in a heterogeneous multi-server environment where scheduling was performed by a central unit. In the mentioned problem, jobs were entered into the system with Poisson distribution, and the service time in each of the service providers was exponential. After entering the jobs into the system, the process of allocating them to the servers was done by the central scheduler, and the goal of the presented model was to minimize the service time for the jobs assigned to each server. Sauer (1998) proposed an approach that considers the modeling and processing of scheduling in the production space that had several separate production sites. In this regard, he stressed that the existence of proper coordination and feedback between the local and global level of production schedule in the factory is very important, which has not been

considered before.

Thoney et al. (2002) suggested a method for simultaneous scheduling of multi-factory supply chains by considering inter-factory transportation, which was used for distributed scheduling problems in the job shop environment. The ultimate goal was to meet the maximum delivery time in the supply chain. Chan et al. (2005), for distributed scheduling problems with the job shop configuration, proposed an adaptive genetic algorithm (GA) with dominated genes, which aims to reduce costs in the entire production system. Naderi & Ruiz (2010) tried to solve the six different models of distributed scheduling problems in a permutation flow shop environment, with heuristic and *meta*-heuristic methods based on local search.

Guo et al. (2015) developed a mathematical model that included three objective functions for the multi-site order planning problem. They considered uncertainty in orders, processing times, and daily production capacity. Furthermore, taking into account learning effects, they presented a *meta*-heuristic harmony search-based Pareto optimization (HSPO) approach and used the Monte Carlo simulation method to solve this problem. Gonzalez-Neira et al. (2017) presented a heuristic method named the biased-randomized simheuristic algorithm, for distributed assembly permutation flow shop scheduling problems considering random processing and assembly time. They have proved that despite their proposed algorithm being designed for a stochastic model, it also can solve a deterministic model.

Marandi & Fatemi Ghomi (2019) introduced a model in which the distributed production scheduling with network configuration was integrated with the vehicle routing problem considering time windows. The topic discussed in this problem was the allocation of jobs to factories, the determination of the sequence of jobs in each factory, and finally, after the completion of the production process, the determination of optimal routes for the distribution of products among customers. They applied an imperialist competitive algorithm (ICA), an improved imperialist competitive algorithm (IICA), particle swarm optimization (PSO), improved particle swarm optimization (IPSO), and GA to tackle the problem. Lei et al. (2020) applied the imperialist competitive algorithm with memory (MICA) for the distributed production scheduling problem with unrelated parallel machines. The model was solved by calculating the total number of machines in all factories and then assigning jobs to them. It is noteworthy that the presented model is implemented for a real case study in the PVC production industry.

Yang & Xu (2021) presented a distributed assembly permutation flow shop scheduling problem with flexible assembly and batch delivery, where the relevant problem has three levels: processing, assembly, and delivery. The set of orders that arrived from customers at zero time included several products, and each product required several jobs to be processed in one of several similar factories. Finally, the job related to a specific product was sent to one of the assembly factories for the assembly phase, each of which had only one assembly machine. Then the final products were batched and delivered to the customers. Also, Lei & Liu (2020) introduced an artificial bee colony method with division (DABC) for the distributed production scheduling problem in parallel machines configuration with unrelated machines. They added preventive maintenance (PM) to the model by assigning time windows to activities related to PM for each machine. Huang et al. (2021) also presented an improved iterated greedy algorithm method based on group thinking (gIGA) for the distributed assembly scheduling problem in the permutation flow shop environment, which had two stages of production and assembly. After allocating jobs to factories and determining the sequence of their processing in each factory, the order of assembling the products, which was done by only one machine, should have been determined.

Fathollahi-Fard et al. (2021) presented a tri-objective model for the distributed scheduling problem in the permutation flow shop environment concerning the sustainability concept, in which the triple bottom line concept (3BL-TBL) was noted. The objective functions of the model were minimizing the makespan and energy consumption and

maximizing social benefits concerning the social indicators of job opportunities and lost working days. Furthermore, they presented an extension of the social engineering optimizer (SEO) method to solve the model.

Hou et al. (2022) presented an integrated mathematical model for the distributed production problem and the distribution problem considering time windows in which a set of jobs (in this case, customer orders) must be allocated to one of the existing factories, which have flow shop configuration to process. Further, after completing the jobs, the products were delivered to the customers who were in different areas using vehicles with a certain capacity, considering meeting the time windows as much as possible. The objective function of this problem has aimed to minimize the total weighted of earliness and tardiness and to solve it; they have developed an enhanced brainstorm optimization (EBSO) algorithm.

Shao et al. (2023) presented a MILP model for the distributed heterogeneous hybrid flow shop scheduling problem with the integration of the lot-streaming problem. They considered the difference in the processing capacity of each factory and the division of each job into several sub-lots to minimize the maximum completion time. They used numerous constructive heuristics and an iterated local search (ILS) algorithm to solve the problem. Gholami & Sun (2023) introduced a new concept in the field of distributed scheduling by introducing the distributed hybrid flow shop scheduling problem with multiprocessor tasks. They considered the minimization of the makespan among distributed production units. They used a conditional Markov chain search to automate the generation of heuristics and used a hybrid Q-learning-local search algorithm to solve it.

On the other hand, in the case of using constraint programming for solving distributed scheduling problems since 1996, scheduling problems have been considered one of the most successful application areas of CP modeling, which has made many software developments and production companies, such as IBM, seek to develop search engines to solve these problems more efficiently (Laborie et al., 2018). Here, a few numbers of papers that deal with the application of the CP in production scheduling problems will be reviewed:

El Khayat et al. (2006) proposed a MILP and a CP model for an integrated production and material handling scheduling problem while considering machines and vehicles as constraining resources. The two proposed models were solved and compared to each other in case of obtaining an optimal solution for used instances. Kelbel & Hanzálek (2011) introduced a use case of constraint programming for production scheduling problems with earliness and tardiness penalties. They conducted a case study of a lacquer production scheduling problem and a job shop scheduling problem considering earliness/tardiness costs and proposed two algorithms to help the constraint programming solver in finding solutions for complex scheduling problems. Gökgür et al. (2018) proposed a CP model for parallel machine scheduling with tool loading to assign a set of predefined jobs to a limited set of tools that are available in due time. The objective function of this problem is minimizing the maximum completion time of jobs and the obtained solutions from the proposed CP model were compared to other solution methods to investigate its applicability for this kind of problem. Novas (2019) introduced a CP model for scheduling problems in a flexible job shop environment considering the lot streaming problem simultaneously to minimize the makespan. The CP model was used for both lots splitting, and the scheduling of production tasks in the proposed model and was solved for some case studies to prove its efficiency in producing solutions with good quality. H'Mida & Lopez (2013) presented a model for the scheduling problem in several production sites with the use of the CP method, which sought to minimize the total completion time of jobs in all factories. To the best of our knowledge, this research was the first attempt at using the CP method for distributed production scheduling problems. Next, Meng et al. (2020) introduced five different models for the distributed production scheduling problem in the flexible job shop configuration. They presented four MILP models, based on sequence,

Table 1

The representative works on the distributed production scheduling problem (Continued).

Author	Year	Shop Configuration	Network Structure			Factory Type		Objective Function	Solution Methods
			Network	Parallel	Serial	Homogenous	Heterogeneous		
Bonomi & Kumar (1990)	1990	Single machine					✓	Service Time	Queuing theory
Sauer (1998)	1998	Parallel machine		✓		✓		Cost	Heuristic
Thoney et al. (2002)	2002	Job Shop			✓	✓		$L_{\max}$	Heuristic
Chan et al. (2005)	2005	Job Shop		2713		✓		$C_{\max}$	GA
Naderi & Ruiz (2010)	2010	Flow Shop		✓		✓		$C_{\max}$	Variable Neighborhood Descent
H'Mida & Lopez (2013)	2013	Job Shop	✓				✓	$\sum C_j$	CP
Guo et al. (2015)	2015	Job Shop		✓			✓	tri-objectives (Tardiness, Throughput time, idle time)	HSPO + Monte Carlo
Gonzalez-Neira et al. (2017)	2017	Flow Shop		✓		✓		$C_{\max}$	Simheuristic
Marandi & Fatemi Ghomi (2019)	2019	Single Machine	✓				✓	Cost (Delivery, Tardiness)	ICA, IICA, PSO, IPSO, GA
Meng et al. (2020)	2020	Job Shop		✓		✓		$C_{\max}$	Heuristics and CP
Lei & Liu (2020)	2020	Parallel machine		✓				$C_{\max}$	DABC
Yang & Xu (2021)	2020	Flow Shop		✓		✓		Cost (Delivery, Tardiness)	Heuristic and meta-heuristic
Lei et al. (2020)	2020	Parallel machine		✓			✓	$C_{\max}$	MICA
Huang et al. (2021)	2021	Flow Shop		✓			✓	$\sum F_j$	gIGA
Fathollahi-Fard et al. (2021)	2021	Flow Shop		✓			✓	tri-objectives (Sustainable development)	SEO
Hou et al. (2022)	2022	Flow Shop		✓		✓		Total weighted of earliness and tardiness	EBSO
Karabulut et al. (2022)	2022	Flow Shop		✓		✓		$C_{\max}$	CP and Heuristics
Shao et al. (2023)	2023	Flow Shop		✓			✓	$C_{\max}$	Heuristics and ILS
Gholami & Sun (2023)	2023	Flow Shop		✓		✓		$C_{\max}$	hybrid Q-learning-local search
Present Study	2023	Flow Shop and Job Shop	✓				✓	$C_{\max}$, Transportation cost	CP

position, time index, and adjacent sequences, as well as a model according to the concepts of CP. Finally, it was found that the constraint programming model both for small-sized and large-sized numerical instances has been more efficient than the other models. After that, the sequence-based model performed better than the other MILP models.

Karabulut et al. (2022) presented a mixed integer linear programming model and a constraint programming model for the distributed scheduling problem considering the blocking constraint in a flow shop environment. These models sought to minimize the maximum completion time of jobs. Several conventional heuristic methods in scheduling problems have been used to solve the model for high-dimensional examples.

The following, summary of the reviewed papers in the field of distributed production scheduling can be seen in Table 1.

Table 1 shows that a significant percentage of the studies done in the field of distributed production scheduling is related to flow shop environments with objective functions related to minimizing the maximum completion time of jobs ($C_{\max}$). On the other hand, in most of the efforts performed in this area, the connection between factories is considered parallel and only confined to one echelon. In other words, the number of papers in which factories are located on several echelons and

considering series or network configurations is very rare. Also, not much attention has been paid to the transportation costs between factories, which is one of the most important criteria for selecting factories to process jobs. Furthermore, according to the investigations carried out for this research, in all the papers, due to the simplification of the assumptions, the shop configuration of the factories in one production network is considered to be completely similar which is in most cases far away from reality.

Regarding the solution method, most of the papers have used heuristic and meta-heuristic methods to solve problems for medium and large-sized instances. On the other hand, exact solution methods and even the new techniques that use these approaches for solving problems like the constraint programming method, have higher reliability in obtained solutions. Nevertheless, due to the difficulty of both implementing and solving these kinds of problems, the exact methods were used in rare cases and only to solve some problems that have parallel structures and one echelon. For this reason, according to the research gap observed in the activities carried out so far, in this research we take a different step to provide a model closer to the real-world limitations to develop these problems. As a result, the contributions of the problem presented in this paper are summarized as follows:

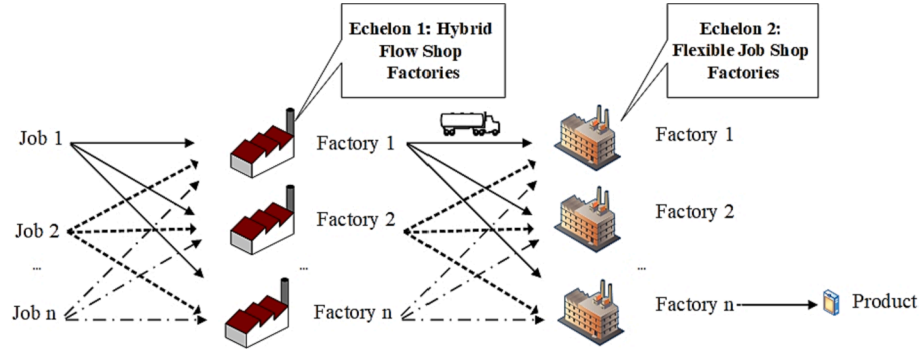


Fig. 1. The network structure of the proposed problem.

- To the best of our knowledge for the first time, we present a mixed integer linear programming model for the distributed production scheduling problem considering a two-echelon network configuration with different shop configurations for factories: first and second echelon factories have hybrid flow shop and flexible job shop configuration, respectively.
- Considering two objective functions of minimizing the maximum completion time of jobs and transportation costs between echelons simultaneously in the distributed production scheduling problem with network configuration.
- Presenting a model according to the concepts of constraint programming technique to solve the proposed model.

3. Problem statement and modeling

In distributed scheduling problems, several jobs must be distributed among various factories to be completed. Therefore, it can also be interpreted that the final product of each factory is the raw material for the next production unit. This characteristic of these groups of problems is similar to assembly stages, but the distributed scheduling problem has its unique property, which is its ability to solve three sub-problems of factory allocation, assignment of operations to machines, and sequencing on machines, simultaneously.

In this paper, the distributed production scheduling problem considering the network configuration for factories in two echelons, in the situation that all factories belong to a central agent is presented. The shop configuration of factories differs from each other in two echelons of the production network, and the transportation costs between selected factories of different echelons are considered.

In the first and second echelons, all factories have a hybrid flow shop and flexible job shop configuration, respectively. Fig. 1 is a schematic figure of the proposed problem:

As shown in Fig. 1, each job is initially assigned to one of the first echelon factories, and all related stages are performed in a hybrid flow shop environment. Then, after completing the stages related to each job in the first echelon, the work-in-process (the final products at the first echelon) is transferred to the second echelon and each job is assigned to one of the factories of that echelon to complete the operations related to it in a flexible job shop configuration. The ultimate goal is to minimize the maximum completion time of jobs and transportation costs throughout the network. It is assumed that due to the almost similar geographical location of the first echelon factories, the criteria influencing the assignment of factories to jobs at that echelon are the processing times and the availability of machines. However, to select a factory in the second echelon, in addition to the criteria mentioned above, the transportation costs of work-in-process between echelons are also considered.

In more detail, in this problem at both echelons, there is a set of jobs $I = \{1, 2, \dots, n\}$ that must be assigned to the set of factories $F = \{1, 2, \dots, p\}$ at each echelon $L = \{1, 2, \dots, e\}$ for processing.

In first-echelon factories that have a hybrid flow shop environment, each job must go through the same route with a set $J = \{1, 2, \dots, k\}$ of stages. In each stage, there are $M = \{1, 2, \dots, q\}$ parallel machines and the number of machines must be at least two. Each job in each factory can be processed on one of the machines from the predefined set MH . In this model, parallel machines in different stages of the hybrid flow shop scheduling problem are considered identical.

In second-echelon factories, where the shop configuration is a flexible job shop, each job involves $J = \{1, 2, \dots, k\}$ operations and has its own processing route. There is also a set MM of identical parallel machines to process each operation of each job in the factories.

3.1. Assumptions

The main assumptions of the research problem are as follows:

- Each job must be assigned to one factory of each echelon for processing; and once a job is assigned to a factory, it can no longer be moved to another factory to continue the process, and all the operations of that job must be completed in the same factory.
- Each machine can process only one job at a time.
- Each job can only be processed on one machine at a time.
- Each factory can process only one job at a time, and one job can only be processed in only one factory at the same time.
- All the jobs and machines are available at time zero.
- All the processing time and transportation costs data are deterministic and known in advance.
- No assumptions have been made regarding the breakdown of the machine or maintenance activities.
- No preemption or interruption is allowed in the processing of jobs on each machine.
- The flow of jobs in factories between different echelons is considered only forward.
- The setup time of the jobs is considered in the processing time corresponding to the same job.
- The transportation cost between selected factories among consecutive echelons is considered.

4. Notations

Considering the dummy job of 0 and $n + 1$, the sets and indices, parameters, and decision variables used in the MILP model are as follows:

Sets and indices

i, i'	Index of jobs where $i, i' \in I$
j, j'	Echelon 1: index of stages where $j, j' \in J$
	Echelon 2: index of operations where $j, j' \in J$
m, m'	Index of machines where $m, m' \in M$

(continued on next page)

(continued)

l	Index of echelons where $l \in L$
f, f'	Indexes of factories where $f, f' \in F$
LO_i	The last stage of each job i in the first echelon
A_{ij}	Set of stages required to process job i at the first echelon
MH_{jm}	Set of eligible machines for each stage j at the first echelon
MM_{ijmfl}	Set of eligible machines for each operation j of job i at the second echelon
FL_{fl}	Set of factories f with distinctive shop configurations for every echelon l

Parameters

pf_{ijmfl}	Processing time of job i at stage j on machine m in factory f of the first echelon
pj_{ijmfl}	Processing time of operation j of job i on machine m in factory f of the second echelon
$cost_{ff'}$	Transportation costs between factory f and f'
N	A large positive number

Decision variables

C_{ijmfl}	Echelon 1: completion time of job i at stage j on machine m in factory f
CT_{ijl}	Echelon 2: completion time of operation j of job i on machine m in factory f
S_{il}	Last completion time of job i in factory f at echelon l
S_{il}	Start time for the first operation of job i at echelon l
C_{max}	Maximum completion time
Y_{ijl}	$Y_{ijl} = \begin{cases} 1, & \text{if job } i \text{ is assigned to factory } f \text{ of echelon } l \\ 0, & \text{otherwise} \end{cases}$
X_{ijmfl}	Echelon 1: $X_{ijmfl} = \begin{cases} 1, & \text{if job } i \text{ at stage } j \text{ is processed by machine } m \text{ in factory } f \\ & \text{of echelon } l \\ 0, & \text{otherwise} \end{cases}$
$W_{ijijmfl}$	Echelon 2: $W_{ijijmfl} = \begin{cases} 1, & \text{if operation } j \text{ of job } i \text{ is processed by machine } m \\ & \text{in factory } f \text{ of echelon } l \\ 0, & \text{otherwise} \end{cases}$
$W_{ijijmfl}$	Echelon 1: $W_{ijijmfl} = \begin{cases} 1, & \text{if job } i \text{ at stage } j \text{ is processed before job } i' \text{ at stage } j' \text{ by} \\ & \text{machine } m \text{ in factory } f \text{ of echelon } l \\ 0, & \text{otherwise} \end{cases}$
$W_{ijijmfl}$	Echelon 2: $W_{ijijmfl} = \begin{cases} 1, & \text{if operation } j \text{ of job } i \text{ is processed before operation } j' \text{ of job } i' \\ & \text{by machine } m \text{ in factory } f \text{ of echelon } l \\ 0, & \text{otherwise} \end{cases}$

4.1. The proposed MILP model

Finally, the proposed mathematical model is as follows:

$$\text{Min } C_{\max} \quad (1)$$

$$\text{Min } \sum_{i>1, j<|I|, f, f', l<|L|} cost_{ff'}(Y_{ijl}Y_{ij'l+1}) \quad (2)$$

s.t.

$$\sum_{f \in FL_{fl}} Y_{ijl} = 1 \forall i, l \quad (3)$$

$$\sum_{m \in MH_{jm}} X_{ijmfl} = Y_{ijl} \forall i > 1, i < |I|, (i, j) \in A_{ij}, f \in FL_{fl}, l = 1 \quad (4)$$

$$\sum_{i, j, i' \neq i} j > 1, (i, j) \in A_{ij}, m \in MH_{jm}, W_{ijijmfl} = X_{ijmfl} \forall i < |I|, (i, j) \in A_{ij}, m \in MH_{jm}, f \in FL_{fl}, l = 1 \quad (5)$$

$$\sum_{i, j, j' \neq j} i < |I|, i \neq i', (i, j) \in A_{ij}, m \in MH_{jm}, W_{ijijmfl} = X_{ijmfl} \forall i > 1, (i, j) \in A_{ij}, m \in MH_{jm}, f \in FL_{fl}, l = 1 \quad (6)$$

$$C_{ijmfl}X_{ijmfl} \geq C_{i, j-1, m'fl}X_{i, j-1, m'fl} + \quad \forall i > 1, i < |I|, (i, j) \in A_{ij}, m \neq m' \\ pf_{ijmfl}X_{ijmfl} + N(X_{ijmfl} - 1) \quad m \in MH_{jm}, m' \in MH_{j-1, m'}, f \in FL_{fl}, l = 1 \quad (7)$$

$$\forall i < |I|, i > 1, i \neq i', (i, j) \in A_{ij}, (i', j') \in A_{ij'} \\ C_{ijmfl} + pf_{ijmfl}W_{ijijmfl} + N(W_{ijijmfl} - 1) \leq C_{ijmfl} \quad j = j', m \in MH_{jm}, m' \in MH_{j-1, m'}, f \in FL_{fl}, l = 1 \quad (8)$$

$$CT_{ijl} \geq C_{ijmfl} \quad \forall i > 1, i < |I|, i, j \in A_{ij}, m \in MH_{jm}, f \in FL_{fl}, l = 1 \quad (9)$$

$$C_{\max} \geq CT_{ijl} \forall i > 1, i < |I|, f \in FL_{fl}, l = 1 \quad (10)$$

$$C_{ijmfl} = 0 \forall i = 1, (i, j) \in A_{ij}, m \in MH_{jm}, f \in FL_{fl}, l = 1 \quad (11)$$

$$C_{ijmfl} = N \forall i = |I|, (i, j) \in A_{ij}, m \in MH_{jm}, f \in FL_{fl}, l = 1 \quad (12)$$

$$S_{i2} = \sum_{m \in MH_{jm}, f \in FL_{fl}} l = 1 \quad C_{ijmfl} \forall i, j \in LO_i \quad (13)$$

$$\sum_{m \in MM_{ijmfl}} X_{ijmfl} = Y_{ijl} \forall i > 1, i < |I|, j, f \in FL_{fl}, l = 2 \quad (14)$$

$$\sum_{i, j, i' \neq i} i > 1, m \in MM_{ijmfl}, W_{ijijmfl} = X_{ijmfl} \forall i < |I|, m \in MM_{ijmfl}, f \in FL_{fl}, l = 2 \quad (15)$$

$$\sum_{i, j, i' \neq i} i \neq i', m \in MM_{ijmfl}, W_{ijijmfl} = X_{ijmfl} \forall i > 1, m \in MM_{ijmfl}, f \in FL_{fl}, l = 2 \quad (16)$$

$$C_{ijmfl}X_{ijmfl} \geq C_{i, j-1, m'fl}X_{i, j-1, m'fl} + \quad \forall i < |I|, i > 1, m \neq m', m \in MM_{ijmfl}, \\ pj_{ijmfl}X_{ijmfl} + N(X_{ijmfl} - 1) \quad m' \in MM_{i, j-1, m'fl}, f \in FL_{fl}, l = 2 \quad (17)$$

$$S_{i2} + C_{ijmfl} + pj_{ijmfl}W_{ijijmfl} \quad \forall i < |I|, i > 1, i \neq i', \\ + N(W_{ijijmfl} - 1) \leq C_{ijmfl} \quad m \in MM_{ijmfl}, m' \in MM_{i, j-1, m'fl}, f \in FL_{fl}, l = 2 \quad (18)$$

$$CT_{ijl} \geq C_{ijmfl} \forall i > 1, i < |I|, j, m \in MM_{ijmfl}, f \in FL_{fl}, l = 2 \quad (19)$$

$$C_{\max} \geq CT_{ijl} \forall i > 1, i < |I|, f \in FL_{fl}, l = 2 \quad (20)$$

$$C_{ijmfl} = 0 \forall i = 1, m \in MM_{ijmfl}, f \in FL_{fl}, l = 2 \quad (21)$$

$$C_{ijmfl} = N \forall i = |I|, m \in MM_{ijmfl}, f \in FL_{fl}, l = 2 \quad (22)$$

$$X_{ijmfl} = 0 \forall i, j, m \in MM_{ijmfl}, f \in FL_{fl}, l = 2 \quad (23)$$

$$C_{ijmfl}, CT_{ijl}, S_{il}, C_{\max} \geq 0 \forall i, j, m, f, l \quad (24)$$

$$X_{ijmfl}, Y_{ijl}, W_{ijijmfl} \in \{0, 1\} \forall i, j, i', j', m, f, f', l \quad (25)$$

Equations (1) and (2) are objective functions that seek to minimize the maximum completion time of jobs in each selected factory and transportation cost for those factories between two echelons, respectively. In general, constraint sets (3) to (13) belong to the hybrid flow shop scheduling model, and constraint sets (14) to (23) are related to the flexible job shop scheduling model. To avoid duplication of explanations, some constraints that work almost identically in both shop configurations are described in one statement.

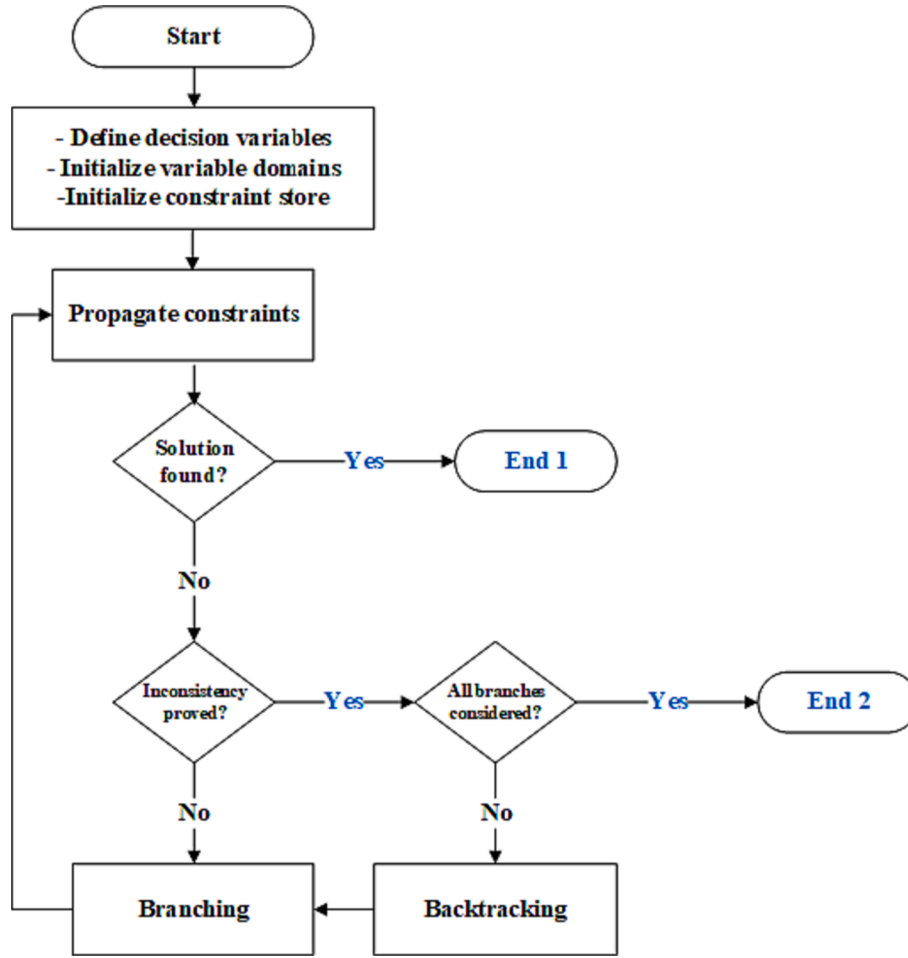


Fig. 2. General algorithm for solving constraint satisfaction problems.

Constraint set (3) indicates that at each echelon, each job should be assigned to only one factory. Constraint set (4) guarantees the fact that each job at each stage must be processed by only one machine, and also states that all stages related to a job must be done in the same factory to which the job was initially assigned.

Constraint sets (5) and (6) guarantee that if a job is scheduled for processing on a machine during a particular stage, other jobs from different stages must be scheduled both before and after it sequentially. Constraint sets (7) and (8) prevent overlapping of the jobs assigned to a machine at each stage and control the fact that the process of each job of any stage can begin only when the processes of the precedence jobs before that stage are totally completed.

Constraint set (14) restricts each operation of every job to be processed by a single machine, and further mandates that all operations belonging to a job must be executed within the same factory where the job was initially assigned. Constraint sets (15) and (16) ensure that if an operation of a job is placed on a machine for processing, it must appear in sequence before one operation from one job and after another operation from a different job. Constraint sets (17) and (18) prohibit any overlapping of operations assigned to a machine, taking into account the requirement that the processing of each operation can only commence once the preceding operations have been completed.

Constraint set (13) defines the interdependence between the completion times of jobs within two echelons. In other words, by calculating the decision variable S_{il} that is used in constraint (18), the fact that the completion time of the first operation of any job at the second echelon must be greater than the completion time of the last stage of the same job at the first echelon control by this constraint.

Constraint sets (9) and (19) in two shop configurations, respectively, indicate the completion time of each job in each factory. Constraint sets (10) and (20) are also presented in this model to define the maximum completion time of each job in each factory. Controlling the completion times of dummy jobs for both the hybrid flow shop and the flexible job shop model is done by constraint sets (11) and (12) as well as (21) and (22), respectively. Constraint set (23) has also been added to the model to specify machines that cannot process jobs in factories with flexible job shop environments for ease of solving the model. Finally, the Constraint sets (24) and (25) indicate the type of variables.

Since there are nonlinear expressions in the proposed model that increase the difficulty of solving the model, using the well-known method proposed by Glover & Woolsey (1974), by defining auxiliary variables and adding new constraints to the model, an attempt has been made to linearize it. Hence for the expression (2), which is one of the objective functions where there is a multiplication of the two binary decision variables, the binary auxiliary variable $U_{iff'l,l+1}$, and also for the right-hand side of constraint sets (7) and (17) which multiply continuous decision variables and binary decision variables, binary auxiliary variable V_{ijmfl} are defined to linearize the model. As a result, the following constraints are added to the proposed model:

$$\text{Min} \sum_{i>1, i<|I|, f, f', l<|L|} \text{cost}_{ff'} U_{iff'l,l+1} \quad (26)$$

$$Y_{ift} + Y_{if',l+1} - 2U_{iff'l,l+1} \geq 0 \forall i < |I|, f, f', l < |L| \quad (27)$$

$$Y_{ift} + Y_{if',l+1} - U_{iff'l,l+1} \leq 1 \forall i < |I|, f, f', l < |L| \quad (28)$$

$$V_{ijmfl} \geq V_{i,j-1,mfl} + Pf_{ijmfl} X_{ijmfl} + N(X_{ijmfl} - 1) \quad \forall i > 1, i < |I|, (i, j) \in A_{ij}, m \neq m' \\ m \in MH_{jm}, m' \in MH_{j-1,m}, f \in FL_{fl}, l = 1 \quad (29)$$

$$V_{ijmfl} \leq C_{ijmfl} \quad \forall i > 1, i < |I|, (i, j) \in A_{ij}, m \in MH_{jm}, f \in FL_{fl}, l = 1 \quad (30)$$

$$C_{ijmfl} - V_{ijmfl} \leq N(1 - X_{ijmfl}) \quad \forall i > 1, i < |I|, (i, j) \in A_{ij}, m \in MH_{jm}, f \in FL_{fl}, l = 1 \quad (31)$$

$$V_{ijmfl} \leq NX_{ijmfl} \quad \forall i > 1, i < |I|, (i, j) \in A_{ij}, m \in MH_{jm}, f \in FL_{fl}, l = 1 \quad (32)$$

$$V_{ijmfl} \geq V_{i,j-1,mfl} + Pj_{ijmfl} X_{ijmfl} + N(X_{ijmfl} - 1) \quad \forall i < |I|, i > 1, m \neq m', m \in MM_{ijmfl}, \\ m' \in MM_{i,j-1,mfl}, f \in FL_{fl}, l = 2 \quad (33)$$

$$V_{ijmfl} \leq C_{ijmfl} \quad \forall i > 1, i < |I|, m \in MM_{ijmfl}, f \in FL_{fl}, l = 2 \quad (34)$$

$$C_{ijmfl} - V_{ijmfl} \leq N(1 - X_{ijmfl}) \quad \forall i > 1, i < |I|, m \in MM_{ijmfl}, f \in FL_{fl}, l = 2 \quad (35)$$

$$V_{ijmfl} \leq NX_{ijmfl} \quad \forall i > 1, i < |I|, m \in MM_{ijmfl}, f \in FL_{fl}, l = 2 \quad (36)$$

$$V_{ijmfl} \geq 0 \quad \forall i, j, m, f, l \quad (37)$$

$$U_{ijf,l+1} \in \{0, 1\} \quad \forall i, f, l \quad (38)$$

According to the explanations provided earlier, expression (2) is replaced by expression (26), and constraint sets (27) and (28) are added to the other constraints of the model for linearization. Also, constraint sets (7) and (17) are removed from the model, and constraint sets (29) and (33), which are the linearized expressions of those, are added to the model along with constraint sets (30) to (32) and (34) to (36). Also, Constraint sets (37) and (38) specify the type of auxiliary variables used for linearization.

5. The CP model

The constraint programming method, with its ability to separate the modeling and search steps to obtain a solution, can define a clearer description of the constraints of a problem, and this is considered one of its main advantages. The basic ideas of constraint programming are derived from concepts related to artificial intelligence and are known as an approach to formulate and solve discrete variable constraint satisfaction problems as well as constrained optimization problems that systematically use inductive reasoning to reduce the search space. In general, each constraint satisfaction problem includes a set of decision variables $X = \{x_1, x_2, \dots, x_n\}$, a set of domains $D = \{d_1, d_2, \dots, d_n\}$ of values that each decision variable may take, and a set of constraints. The main goal in solving such problems is to assign values to the decision variables using the domain defined on them so that all the constraints in the problem are met (Kanet et al., 2004); (Shahmoradi et al., 2017).

To be more precise, to solve a problem by the constraint programming method, it is necessary that the mathematical model of the problem be formulated as a constraint satisfaction problem (CSP) and it is at the heart of CP, so it can be rewritten by one of the constraint programming languages (Kanet et al., 2004). Next, to transfer information between constraints and variables, the constraint programming method uses constraint propagation to prevent the solution from becoming impossible (Gedik et al., 2016).

Fig. 2 illustrates the general algorithm to solve CSP. The algorithm starts with the definition of variables, their domains, and constraints. At the beginning of the process, all constraints are stored in an initial constraint store. The next step is called constraint propagation and domain reduction. During this process, a variable along with its domains

is selected as an input, and according to the concept of local consistency, an attempt is made to remove values from the domain that are by no means the final solution. In other words, by reducing the domains, it keeps only possible and feasible solutions. Then, two possibilities arise; the first one is that a solution is found, in which the search algorithm terminates (End 1). Another situation is that no solution can be found in this step. In this case, the inconsistency of the problem should be checked at the current stage. Local consistency, which is one of the most important concepts in solving a constraint programming problem, investigates the fact that there are no local inconsistencies for the constraints. Local inconsistency is a situation where some constraints are such that only some variables of the problem apply to them, but they cannot be generalized to one or more variables of the problem. If the existence of inconsistency is proven, a failure has occurred. For this purpose, in the next step, the search tree is checked to determine whether all its branches have been considered or not. If all the branches have been checked, the problem is proved to be inconsistent (End 2). Otherwise, the algorithm returns to the previous step and examines another subproblem that has not yet been considered. On the other hand, if the inconsistency is not proven, the search is performed by re-branching for the desired problem. Repeating this process on the constraints continues until either the variable domains cannot be reduced or a domain becomes empty and so-called a failure occurs, in which case the model will have no solution (CSP, 2020); (Laborie et al., 2018); (Rossi et al., 2006). Finally, it comes to the search stage where the constraint solvers examine the solution space with one of the search algorithms such as backtracking, branch and bound algorithm, or even different types of local search to finally solve the relevant problem and report its results (Rossi et al., 2006).

With these interpretations, even though every constraint satisfaction problem provides a feasible solution in which all constraints are satisfied, there is no guarantee of providing an optimal solution. Therefore, another problem called the constrained optimization problem (COP) is presented to solve this issue. In the constrained optimization problem, just like the constraint satisfaction problem, there is a set of decision variables, a set of domains of values that each decision variable may take, and a set of constraints. Also, in addition to these, an objective function is defined for the problem so that by taking it into account and solving the model, an allocation of the values of the domains to the variables is provided on the condition that all the constraints are met and the objective function is optimized. In other words, this problem obtains a feasible solution that is also optimal (Kanet et al., 2004).

The algorithm for solving the constrained optimization problem is based on the approach of solving the constraint satisfaction problem, which will be briefly explained below: for example, to solve a constrained optimization problem that has a minimization objective function, when the first solution for the main satisfaction problem is found, its target value is calculated (Z'). This value is written as a $Z < Z'$ constraint and is added to the other constraints of the constraint satisfaction problem solved at the same stage to form a new constraint satisfaction problem. So, the solution algorithm is repeated according to any other constraint satisfaction problem until the final solution that is the optimal solution itself is reached (Kanet et al., 2004).

With regard to these explanations, to solve the problem presented in this research, the constraint programming model of this problem has been written using interval decision variables and sequence decision variables, which are among the most famous and widely, used decision variables in the field of constraint programming. The indices and sets, parameters, decision variables, and functions used in the CP model are as follows:

Sets and Indices

i	Index of jobs where $i \in I = \{1, 2, \dots, n\}$
j	Echelon 1: Index of stages where $j \in J_1 = \{1, 2, \dots, k\}$ Echelon 2: Index of operations where $j \in J_2 = \{1, 2, \dots, k\}$

(continued on next page)

(continued)

mf	Index of machines at the first echelon where $mf \in MF = \{1, 2, \dots, qf\}$
mj	Index of machines at the second echelon where $mj \in MJ = \{1, 2, \dots, qj\}$
ff	Index of factories at the first echelon where $ff \in FF = \{1, 2, \dots, pf\}$
fj	Index of factories at the second echelon where $fj \in FJ = \{1, 2, \dots, pj\}$
$jlastf_i$	Set of last stages of each job i at the first echelon
$jlastj_i$	Set of last operations of each job i at the second echelon
$jfirstj_i$	Set of first operations of each job i at the second echelon

The calculation expressions of $jlastf_i$, $jlastj_i$, and $jfirstj_i$ sets are fully described in the CP code.¹

Parameters

$stld$	The number of each step of the first-echelon jobs indicates the position of that step among all the steps required to perform all the jobs of the same echelon.
$opld$	The number of each step of the second-echelon jobs indicates the position of that step among all the steps required to perform all the jobs of the same echelon.
$Stg_{stld,i,j}$	Stage j of job i at the first-echelon factories
$Ops_{opld,i,j}$	Operation j of job i at the first-echelon factories
SO_{ij}	Echelon 1: Stage j of job i Echelon 2: Operation j of job i
$cost_{ff_i,j_i,j_{i+1}}$	Transportation costs between factory ff and fj
$Modesf_{stld,ff,mf,pf}$	All possible values and modes of eligible factories, available machines, and the processing times to process stage j of job i at the first-echelon factories.
$Modesj_{opld,fj,mj,pj}$	All possible values and modes of eligible factories, available machines, and the processing times to process operation j of job i at the second-echelon factories.

$$\left(\sum_{i \in I} \sum_{mf \in MF} \sum_{mj \in MJ} \sum_{SO_{ij} \rightarrow stld_{ij}, SO_{ij} \rightarrow jlastf_i, SO_{ij+1} \rightarrow opld_{ij+1}, SO_{ij+1} \rightarrow jfirstj_i} \text{cost}_{ff_i,j_i,j_{i+1}}^* \text{presenceOf} \left(\text{modesf}_{stld_{ij}} \right)^* \text{presenceOf} \left(\text{modesj}_{opld_{ij+1}} \right) \right) \quad (40)$$

Decision variables

$stg_{i,j}$	Interval variable of each stage $Stg_{stld,i,j}$ at the first echelon
$ops_{i,j}$	Interval variable of each operation $Ops_{opld,i,j}$ at the second echelon
$modesf_{stld}(\text{optional})$	Optional interval variable for different modes of $Modesf_{stld,ff,mf,pf}$ parameter in the first-echelon factories, includes the start, processing, and end time of the stage assigned to that interval. It should be noted that only one interval should be selected for each stage of each job.
$modesj_{opld}(\text{optional})$	Optional interval variable for different modes of $Modesj_{opld,fj,mj,pj}$ parameter in the second-echelon factories includes the start, processing, and end time of the operation assigned to that interval. It should be noted that only one interval should be selected for each operation of each job.
$mchsf_{mf}$	Sequence decision variable that shows an order of the set of $modesf_{stld}$ interval variables that can be assigned to the machine mf at the first echelon.
$mchsj_{mj}$	Sequence decision variable that shows an order of the set of $modesj_{opld}$ interval variables that can be assigned to the machine mj at the first echelon.

Functions

$endOf$	If there is an interval variable in the final solution, this function returns its end time, otherwise, it is 0 by default.
---------	--

(continued on next column)

(continued)

$startOf$	If there is an interval variable in the final solution, this function returns its start time, otherwise, it is 0 by default.
$alternative$	This function chooses the best possible option among the possible options that exist in the optional interval decision variables $modesf_{stld}$ and $modesj_{opld}$ for each echelon.
$noOverlap$	This function prevents overlapping on a machine for all interval variables assigned to each machine that is present in the final solution.
$endBeforeStart$	This function guarantees that if an interval variable takes a value, the current activity can never be performed before the end of its precedence activity; In other words, it controls the determined sequence for the stages or operations of each job.
$presenceOf$	It is a binary function, which takes the value of 1 if an interval decision variable is present in the final solution, and 0 value otherwise.

Finally, the constraint programming model implemented and solved by OPL programming language in CPLEX software is as follows:

Minimize

$$\begin{aligned} \max \quad & endOf(modesj_{opld}) \\ & \forall SO_{i,k} \rightarrow opld, SO_{i,k} \in jlastj_i, i \in I, j \in J_i, \\ & \quad ff \in FJ, mj \in MJ \end{aligned} \quad (39)$$

$$\forall j \in J_i, j < |J_i|, ff \in FF, fj \in FJ$$

The objective function (39) is written to minimize the maximum completion time of jobs in the second-echelon factories considering the maximum completion time of the jobs in the first-echelon factories. The objective function (40) also minimizes the transportation costs between the selected factories of two echelons.

$$\forall SO_{i,j} \rightarrow stld, i \in I, j \in J_i, alternative(stg_{i,j}, modesf_{stld}) \quad (41)$$

$$\forall mf \in MF, noOverlap(mchsf_{mf}) \quad (42)$$

$$\forall i \in I, j \in J_i, j < |J_i|, endBeforeStart(stg_{i,j}, stg_{i,j+1}) \quad (43)$$

$$\begin{aligned} & \forall SO_{i,j} \rightarrow stld, SO_{i,j+1} \rightarrow stld, \sum_{mf \in MF} \text{presenceOf} \left(\text{modesf}_{stld_{ij}} \right) \times ff = \\ & i \in I, j \in J_i, j < |J_i|, ff \in FF \sum_{mf \in MF} \text{presenceOf} \left(\text{modesf}_{stld_{i,j+1}} \right) \times ff \end{aligned} \quad (44)$$

$$\begin{aligned} & \forall SO_{i,j+1} \rightarrow opld, SO_{i,j+1} \in jfirstj_i \\ & SO_{i,j} \rightarrow stld, SO_{i,j} \in jlastf_i, startOf(ops_{i,j+1}) \geq endOf(stg_{i,j}) \\ & i \in I, j \in J_i, j < |J_i| \end{aligned} \quad (45)$$

$$\forall SO_{i,j} \rightarrow opld, i \in I, j \in J_i, alternative(ops_{i,j}, modesj_{opld}) \quad (46)$$

$$\forall mj \in MJ, noOverlap(mchsj_{mj}) \quad (47)$$

$$\forall i \in I, j \in J_i, j < |J_i|, endBeforeStart(ops_{i,j}, ops_{i,j+1}) \quad (48)$$

¹ - The CP code is available at: <https://github.com/semami60/DSP/blob/main/CPmodel.rar>.

$$\forall SO_{ij} \rightarrow opld, SO_{ij+1} \rightarrow opld, \sum_{mj \in MJ} presenceOf(modesj_{opld_{ij}}) \times ff = \sum_{mj \in MJ} presenceOf(modesj_{opld_{ij+1}}) \times ff \quad (49)$$

Constraint set (41) guarantees that each stage of each job is assigned to exactly one factory and one of the eligible machines of the same factory at the first echelon. Constraint set (42) controls that there is no overlap for stages assigned to a machine at the first echelon. Constraint set (43) controls the precedence relationships between stages related to the same job at the first echelon. Constraint set (44) also guarantees that all the stages related to a particular job must be assigned to only one of the first-echelon factories. Constraint set (45) guarantees that the completion time of the first operation of any job at the second echelon is greater than the completion time of the last stage of the same job at the first echelon. Furthermore, constraint set (46) controls that each operation of each job at the second echelon is assigned to exactly one of the factories of the same echelon and one of the eligible machines from that factory. Constraint set (47) ensures that there is no overlap for operations assigned to a machine at the second echelon. Constraint set (48) also controls the precedence relationships between operations related to a job at the second echelon. Finally, the constraint set (49) guarantees that all operations related to a job must be assigned to only one of the second-echelon factories.

6. Epsilon constraint method

The epsilon constraint method is one of the most widely used approaches to face multi-objective problems, which was proposed by Haimes (1971) in 1971. In fact, this approach tries to solve multi-objective problems by keeping one of the objective functions, which is usually the most important objective function of the problem, and transferring other objective functions to the constraints section at each stage to limit them according to a specific value determined by the user (Deb, 2014). To express this approach more clearly, consider the general multi-objective formulation (50) (Mavrotas, 2009):

$$\left(\sum_{i \in I} \sum_{mj \in MF} \sum_{mj \in MJ} \sum_{SO_{ij} \rightarrow stld_{ij}} \sum_{SO_{ij} \rightarrow Jlastfj} \sum_{SO_{ij+1} \rightarrow opld_{ij+1}} \sum_{SO_{ij+1} \rightarrow Jfirstji} cost_{ff_{ij}ff_{ij+1}}^* presenceOf(modesf_{stld_{ij}})^* presenceOf(modesj_{opld_{ij+1}}) \right) \leq \varepsilon \quad (53)$$

$$\min_{\substack{st \\ X \in S}} (f_1(x), f_2(x), \dots, f_p(x)) \quad (50)$$

In formulation (50), x is the vector of decision variables and $f_1(x), f_2(x), \dots, f_p(x)$ are the objective functions. The feasible region of the problem is shown by S . By implementing the epsilon constraint approach for the mentioned model, formulation (51) is obtained:

$$\begin{aligned} \min_{\substack{st \\ X \in S}} f_1(x) \\ f_2(x) &\leq \varepsilon_2 \\ f_3(x) &\leq \varepsilon_3 \\ &\dots \\ f_p(x) &\leq \varepsilon_p \end{aligned} \quad (51)$$

here, by keeping the first objective function (the main objective function) and converting the other objective functions into constraints at each stage (sub-objective functions), a limit for the epsilon value is

determined. Then by limiting the sub-objective functions according to the obtained epsilon values at each stage, the Pareto solutions of the problem are obtained. In general, the steps of implementing the epsilon constraint method for a multi-objective model can be summarized in the following steps (Mavrotas, 2009):

- 1- One of the objective functions (the objective function that is more important according to the nature of the main purpose of the problem) is selected and maintained as the main objective function.
- 2- At each stage, according to one of the remaining objective functions, the problem is solved by both minimization and maximization objectives, so that the intervals for epsilon are determined by obtaining the optimal values of each.
- 3- The distance between the two optimal values of the sub-objective functions is divided into a predetermined number and the epsilon values are recorded in the compiled table to present the results.
- 4- The desired problem is solved at every stage with the main objective function and limiting the other objective functions by each of the epsilon values.
- 5- Finally, the set of Pareto solutions is reported.

According to the above explanations, to use the epsilon constraint approach for solving the bi-objective model presented in this research, the first objective function, which seeks to minimize the maximum completion time of jobs, is considered the main objective function. In the following, to obtain the epsilon values for each example of the problem, this problem is solved only by considering the second objective function, which is related to the transportation costs between two-echelon factories, once as minimization and once as a maximization problem. Finally, by keeping the first objective function and transferring the second objective function to the constraints section, expressions (52) and (53) are added to the presented mixed integer linear programming and constraint programming models, respectively:

$$\sum_{i>1, i<|I|, f, f', J<|J|} cost_{ff'}^* U_{ff'f', J+1} \leq \varepsilon \quad (52)$$

$$\forall j \in J_i, j < |J_i|, ff \in FF, ff \in FJ$$

7. Numerical results

In this section, first, we explain how to make numerical instances. In the following, the process of solving a small example is described. In the end, the performance of the two presented MILP and CP models is compared by analyzing the computational results.

7.1. Data generation

60 numerical instances in small, medium, and large sizes are randomly generated to examine the proposed MILP and CP models. Regarding the classification of instances based on the number of jobs, it is important to mention that the proposed model's complexity is substantial due to its unique characteristics. These characteristics include considering a two-echelon network configuration environment and different shop configurations for factories. Consequently, it is evident

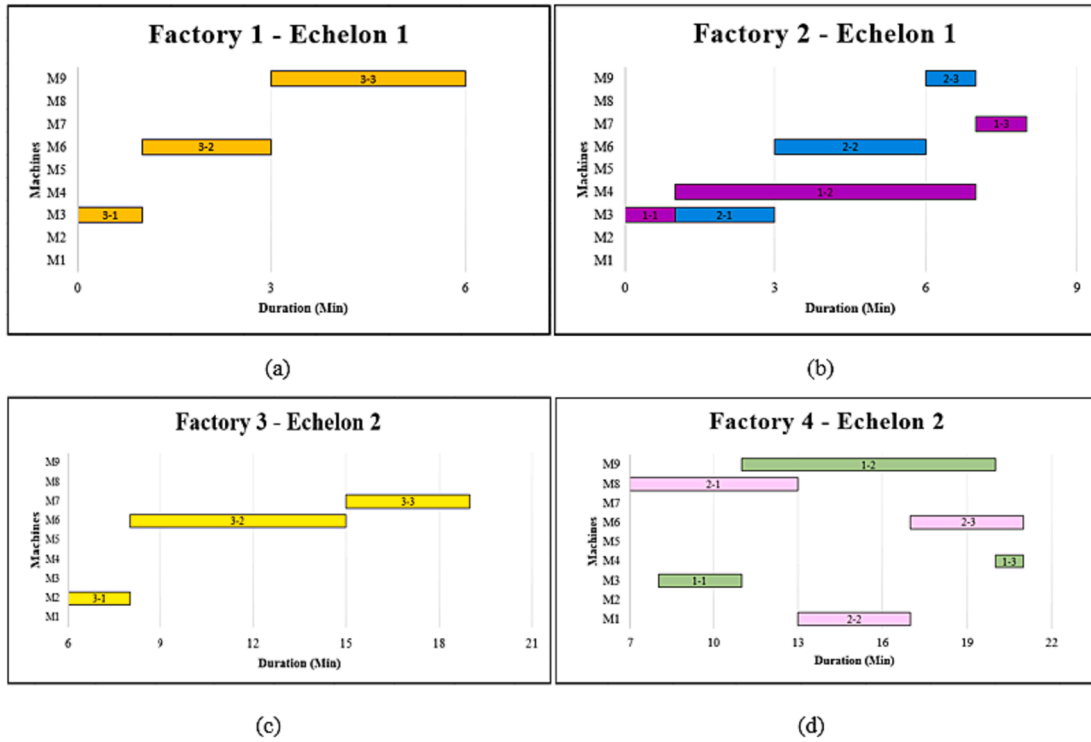


Fig. 3. The Gantt charts for stages/operations of jobs in the network (small instance solved by the MILP model).

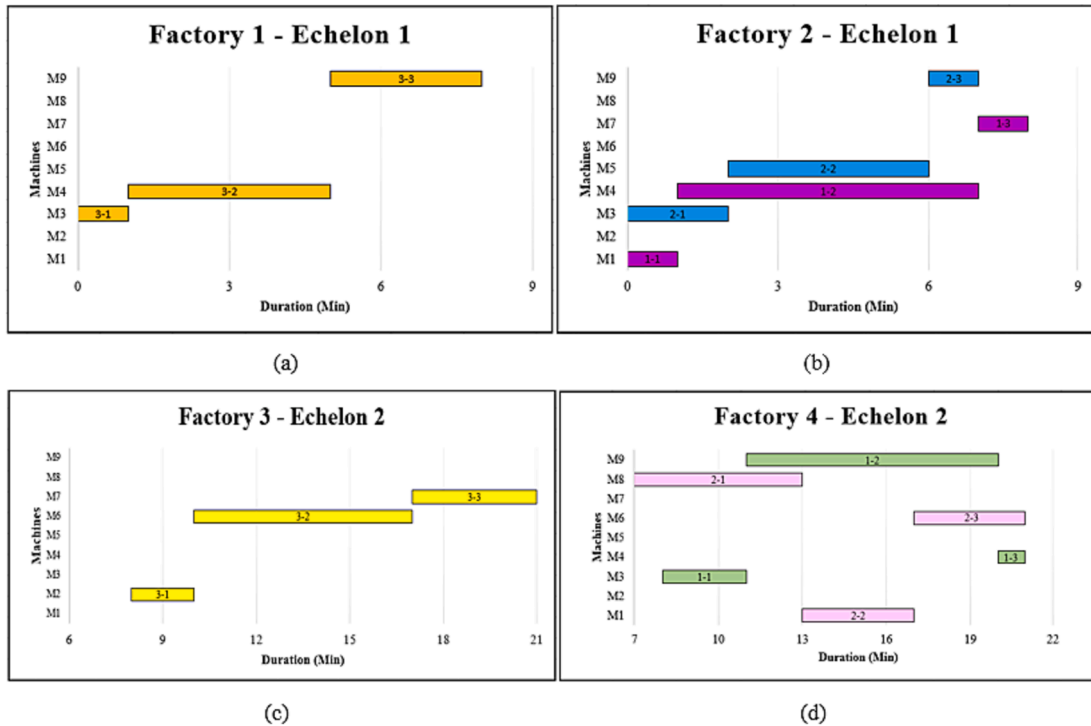


Fig. 4. The Gantt charts for stages/operations of jobs in the network (small instance solved by the CP model).

that an increase in the number of jobs significantly impacts the runtime. After conducting numerous tests and trials, taking into account relevant papers from the literature review, it was determined that instances with

30 jobs can be classified as large-sized for our problem. Each instance is run three times with different values of the parameters to obtain reliable estimates and results, so 180 instances are solved with the MILP² model

² - The MILP code is available at: <https://github.com/semami60/DSP/blob/main/MILPmodel.rar>.

Table 2

The solutions obtained from solving a small instance with different epsilon values.

Iteration	ϵ	The first objective function value	The second objective function value	GRC	Rank
1	36	30	36	0.375	2
2	41	21	41	0.467	1
3	46	21	44	–	–
4	51	21	43	–	–

in GAMS software and the CP model in CPLEX software, separately.

The dimensions of the instances and the values of the parameters used to generate numerical instances in this paper are as follows. Note that the values are taken from the literature (Brandimarte, 1993); (Fernandez-Viagas and Framinan, 2020); (Marandi and Ghomi, 2019).

- The number of jobs is considered as {3, 5, 10, 20, 30}.
- The number of stages/operations for each shop configuration is considered as {3, 5, 10}.
- The number of machines for each stage/operation is considered as {3, 6}.
- The number of factories at each echelon is considered as {2, 4}.
- The processing time parameter is randomly generated using a uniform distribution between [1,10] minutes. The transportation cost parameter is randomly generated using a uniform distribution between [10,20] monetary units.

7.2. GAMS and CPLEX settings

As stated earlier, the MILP model is implemented in GAMS IDE 24.1.2 software using the CPLEX solver. On the other hand, the CP model is implemented in CPLEX Studio IDE 12.9.0 software using CP optimizer. The generated instances are executed on a computer with 12 GB of RAM and a 2.4 GHz CPU. Moreover, a time limit of 3600 s and the zero relative optimality gap is considered to solve the instances.

7.3. An example

In this subsection, we examine the proposed MILP and CP models by solving a small instance. This small instance has three jobs, three stages/operations for each job in each factory, three machines to perform each stage/operation in each factory, and two factories at each echelon. It should be noted that the range of epsilon changes to solve this small instance is calculated between (36, 51), and in this particular example, the value of epsilon is considered equal to 40. The values of the other parameters are shown in Tables 1E-10E.³ The results of solving this small instance can be observed in Fig. 3 and Fig. 4:

As it is clear, the top and bottom charts in both Figures, (a) and (b) charts, correspond to the jobs assigned to the first and second factories, and the (c) and (d) charts indicate that for the third and fourth factories from the second echelon. A summary of the results from solving this numerical instance for both models can be expressed as follows:

- By solving the MILP and CP models, the assignment of jobs to factories is completely similar at the two echelons. In fact, the first and second jobs at the first echelon are assigned to the second factory, and the third job at the first echelon is assigned to the first factory. Also, the first and second jobs at the second echelon are assigned to the fourth factory, and the third job at the second echelon is assigned to the third one. Therefore, the transportation costs between the two echelons in both cases are similar and equal to 37 monetary units.

Table 3

The average of relative difference values of the first objective function based on the results of Table 7.

Group	1st iteration	2nd iteration	3rd iteration
1	–3.21548	–9.8124	–1.45725
2	–6.8835	–17.193	–0.244
3	–9.175	–2.067	–7.498
4	–0.511	–26.0638	–1.2275
5	–0.218	–24.8192	–33.3236
6	–2.55667	–26.253	–0.42833
7	–0.13333	–0.7842	–0.10133
8	–0.40333	–8.4122	–14.0733
9	–0.93633	–0.29133	–5.955
10	–0.8385	–2.893	–60.5392
11	–19.4925	–33.8308	–43.9546
12	–1.24667	–1.104	–4.45067

- On the other hand, the assignment of stages or operations of each job to machines in different factories on two echelons is completely similar in some cases and different in other cases, but the value of the primary objective function of the problem that is the minimization of makespan, in both cases equal to 21, which proves the accuracy of the performance of both models.

Next, to indicate the performance of the epsilon constraint method in solving this bi-objective model, the specific range determined for the epsilon values in the mentioned small example is divided into four equal parts and this example is solved for different epsilon values to obtain Pareto solutions. The results of which are shown in Table 2:

According to Table 2, only the solutions obtained from the first and second iterations are considered the Pareto solutions for this solved example. As it is known, the objective functions of the selected instance in the first iteration for epsilon with a value of 36 are equal to 30 and 36, respectively, and in the second iteration for epsilon with a value of 41, they are equal to 21 and 41. Obviously, none of them can be considered as the dominant solution.

It is worth mentioning that for comparing Pareto solutions and ranking the solutions in more sophisticated instances, we apply the Gray Relational Analysis (GRA) method. According to Wang & Rangaiah (2017) and Wang et al. (2020), GRA is an appropriate algorithm for ranking solutions in a Pareto front. In this method, the gray relational coefficient (GRC) is utilized to describe the similarity between each candidate network (i.e., objective values of each optimal solution) and the best reference network (i.e., an ideal network formed by selecting the best value of each objective). The four stages of this approach are data normalization, defining the ideal point, finding the difference between the normalized objective values of Pareto-optimal solutions and the ideal point, and computing GRC. We applied the GRA method to compare the Pareto solutions by considering the GRC values in Table 2 and ranked the solutions based on the larger values.

7.4. Comparing the performance of the MILP and CP approaches

The results of solving all 180 instances with two approaches are reported in Table 5 and Table 6 in the appendix. The results from solving all the generated instances for this study, including the resulting values for the objective functions, solution time, and solution gap of each instance for a random value of epsilon in the allowed interval, can be seen for two models in Table 5.

The objective function values of those instances marked as “No Solution Returned” indicate no integer solution was found within the time limit of 3600 s for that particular instance. As it is clear, the MILP model with a time limit of 3600 s, only for instances with small-sized and at most 5 jobs, even in the condition that the rest of the dimension of the instance, including the number of stages/operations, machines, and factories, have taken the lowest value, can solve the model with zero solution gap. In other cases, it provides a solution with a gap much

³ - The Tables 1E-10E are available at: <https://github.com/semami60/DSP/blob/main/Example.docx>.

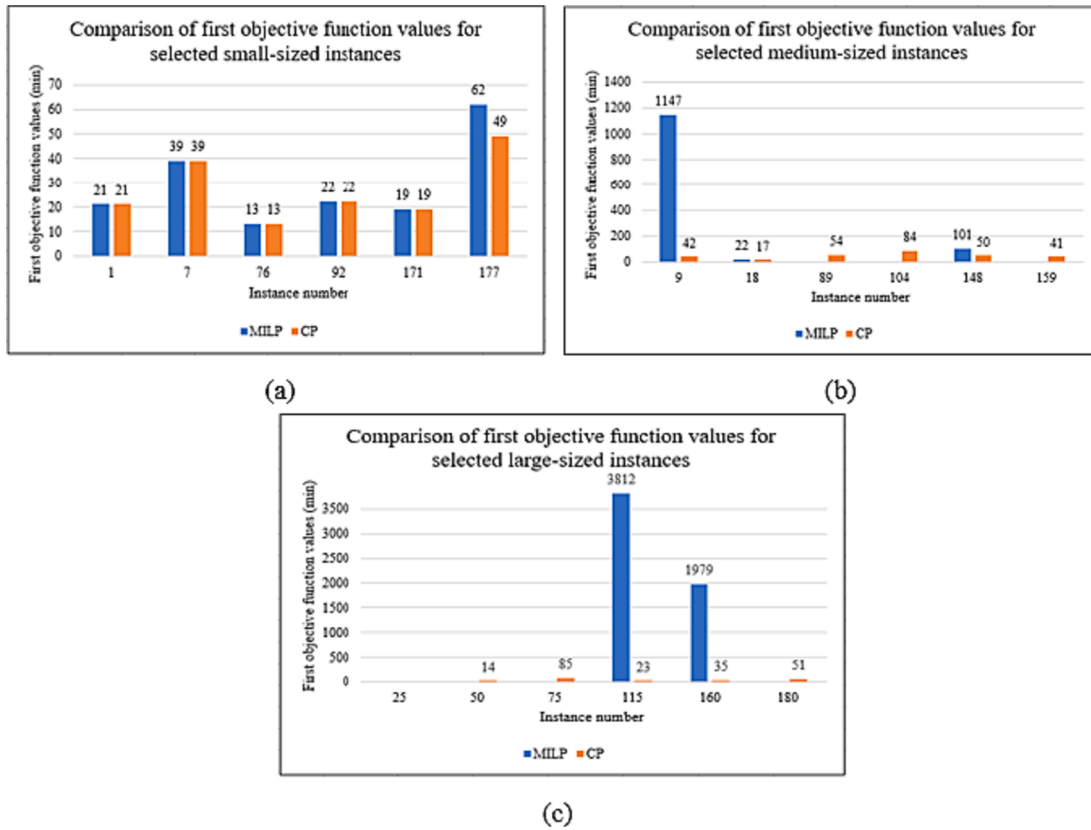


Fig. 5. Comparison of the first objective function values.

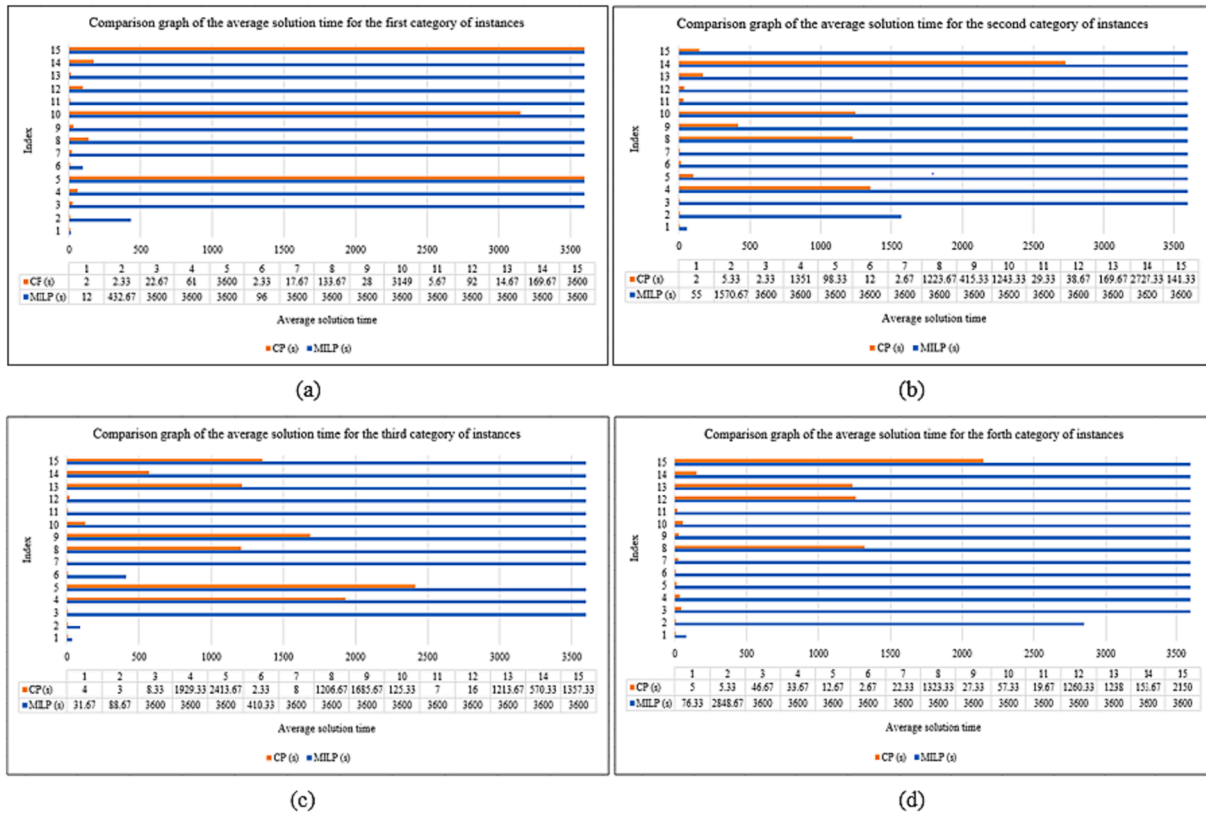


Fig. 6. Comparison of the average solution time for the two models.

Table 4

Summary of the results of solving all numerical instances for the MILP and CP approaches for three iterations.

Iteration	MILP			CP		
	No. solved instances with the optimal solution	No. unsolved instances in 3600 s	No. solved instances with a gap in 3600 s	No. solved instances with the optimal solution	No. unsolved instances in 3600 s	No. solved instances with a gap in 3600 s
1	9	16	35	48	2	10
2	9	8	43	54	0	6
3	9	13	38	55	2	3
Sum	27	37	116	157	4	19
Average	9	12.33	38.67	52.33	1.33	6.33

higher than the optimal solution. On the other hand, the CP model has a significant ability to solve large-sized instances, even with 30 jobs, and provides optimal solutions in a reasonable time. Only a few instances that are solved with the CP approach have a solution gap with a considerably smaller solution gap than the MILP model. In the following, since all the parameters of the instances were generated randomly, to have an accurate and reliable estimate of the solution time for each of them by each software, an average of the results obtained in three iterations is shown in Table 6. The solution time cases marked in bold are the instances that do not give an optimal solution in 3600 s, and the red bold cases are the instances that do not get any integer solution within 3600 s.

Next, in Table 7 in the Appendix, the solutions related to the first objective function which are obtained from solving each numerical instance with the MILP and CP model are compared. These instances have been divided into 12 different groups and all the details concerning each of them can be found in Table 7. Note that the values of the last three columns of this table called the relative difference of the first objective function which is the main objective function of the considered problem, are also obtained from equation (54):

$$\frac{\text{cmax}_{CP} - \text{cmax}_{MILP}}{\text{cmax}_{CP}} \quad (54)$$

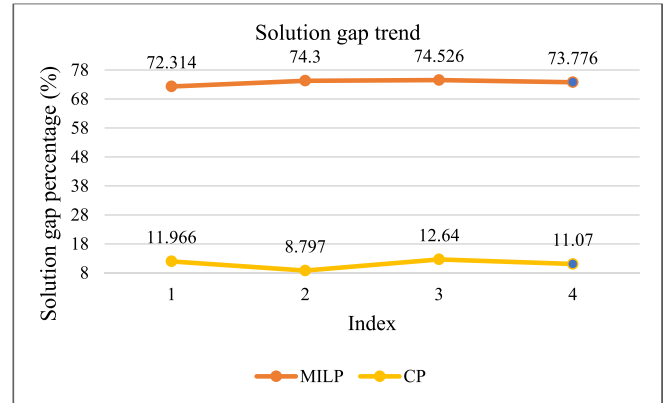
Obviously, the zero value in the last three columns of Table 7 indicates that the MILP and CP models yielded the same value for the first objective function. In only one case in group (3), this numerical value is positive, demonstrating that the MILP model achieved a more satisfactory solution than the CP model. However, in other cases, this number is negative, suggesting that the solutions obtained by the CP model are superior to those obtained by the MILP model.

Table 3 illustrates the average of the obtained results in Table 7 for all 12 groups. As can be seen, all the averages of the relative differences values are negative. This demonstrates that the CP model has good performance in solving the considered problem. It should be noted that naturally, the smaller absolute value of the resulting negative number indicates that the two models have achieved solutions with close gaps.

In order to more closely examine the solutions of the first objective function as the main objective function of the problem, for each dimension of the generated instances (small-, medium- and large-sized), six instances are randomly selected and then the results obtained from solving the two MILP and CP models are analyzed:

In Fig. 5, the numbers written above each column specify the value of the first objective function which is the result of solving each instance with the corresponding model. The number of each numerical instance specified on the horizontal axis is also recorded according to Table 7, to identify the exact characteristics of the desired instance. According to part (a) of Fig. 5, it can be clearly seen that for small instances with just 3 and 5 jobs, for 5 out of 6 selected cases, both models obtained identical solutions.

Part (b) of Fig. 5, which shows the results of solving medium-sized samples with 10 and 20 jobs, represents that the two models achieved close solutions for only one instance, and in other cases, there is a significant difference in the obtained objective function values. In fact, for three instances, the MILP model did not get any integer solution within

**Fig. 7.** Solution gap trends for MILP and CP models.

the 3600-second limit, and in the rest of them, it did not necessarily attain an optimal solution. Finally, according to part (c) of Fig. 5, which is related to the results of solving large-sized instances with 30 jobs, the difference between the solutions obtained from the two models is very strong and cannot be ignored. On the other hand, the MILP model does not provide any integer solution in 3600 s for 4 out of 6 selected instances, and in the other 2 cases, the resulting solution is not optimal. This is even though the CP model could not obtain the solution in the applied time limit for only one of the selected instances (Instance 25), and it solved the rest of the numerical instances optimally.

In general, it can be concluded that with the increase in the dimensions of the problem and the growth in the number of jobs, there is a significant difference in the solutions obtained from the two proposed models, and the CP model performs far better than the MILP model in terms of finding the optimal solution.

Next, the average run time of the instances by two models for three iterations is compared by presenting bar graphs in Fig. 6. To facilitate graphing and interpretation of the results, the instances are divided into 15 groups according to the number of stages/operations, the number of machines, as well as the number of factories at each echelon for different values of jobs. In fact, the number of stages/operations and jobs for the 15 existing instances of each category changes according to the determined sets {3, 5, 10} and {3, 5, 10, 20, 30} respectively, and the number of machines for each stage/operation and factories at each echelon is mentioned separately for each diagram.

In part (a) of Fig. 6, all 15 groups have three machines per stage/operation and two factories per echelon. The blue bars show the average run time to solve each instance by the MILP model in GAMS software, and the orange bars show the average run time to solve each instance by CP in CPLEX software. The table below each graph in Fig. 6 represents the average run time for each instance with the corresponding model. Clearly, the CP model obtained the optimal solution for all instances except for the 15th one, which includes 30 jobs and 10 stages/operations, within the time limit of 3600 s. Meanwhile, the MILP model reached the optimal solution only for three instances with a small value for jobs and achieved the optimal solution in a longer time than the CP

Table 5
The results of solving 180 numerical instances for the two models.

Instance	No. jobs per echelon	No. stages/ operations per echelon	No. machines for each stage/ operation	No. factories per echelon	MILP		Solution Time (s)	Relative gap (%)	CP		Solution Time (s)	Relative gap (%)	ε	Range ε
					cmax	trCost			cmax	trCost				
1st Iteration														
1	3	3	3	2	21	37	11	0	21	37	2	0	40	(36,51)
2	5	3	3	2	23	58	729	0	24	58	3	0	61	(55,78)
3	10	3	3	2	41	182	3600	74	27	179	3	0	182	(163,199)
4	20	3	3	2	133	260	3600	88	27	264	36	0	264	(260,277)
5	30	3	3	2	469	334	3600	96	37	334	3600	16.22	334	(310,556)
6	3	5	3	2	38	43	73	0	38	43	3	0	44	(36,54)
7	5	5	3	2	39	67	3600	5	39	67	37	0	67	(60,95)
8	10	5	3	2	89	146	3600	69	40	150	11	0	158	(116,184)
9	20	5	3	2	1147	371	3600	98	42	336	24	0	371	(286,397)
10	30	5	3	2	No Solution Returned	No Solution Returned	3600	–	49	403	3600	10.2	411	(338,526)
11	3	10	3	2	87	40	3600	25	86	36	3	0	42	(36,54)
12	5	10	3	2	110	65	3600	50	85	65	16	0	65	(63,70)
13	10	10	3	2	254	152	3600	87	79	167	19	0	167	(152,184)
14	20	10	3	2	2709	290	3600	98	77	292	47	0	354	(226,368)
15	30	10	3	2	4707	414	3600	99	No Solution Returned	No Solution Returned	3600	–	418	(406,502)
16	3	3	6	2	20	45	47	0	20	45	2	0	46	(33,60)
17	5	3	6	2	12	70	155	0	12	70	12	0	70	(60,99)
18	10	3	6	2	22	156	3600	63	17	154	3	0	162	(150,170)
19	20	3	6	2	44	248	3600	84	16	246	3600	12.5	249	(200,396)
20	30	3	6	2	No Solution Returned	No Solution Returned	3600	–	17	379	215	0	379	(330,480)
21	3	5	6	2	32	43	3600	46	32	44	16	0	45	(39,54)
22	5	5	6	2	24	72	3600	66	24	70	4	0	74	(50,100)
23	10	5	6	2	43	119	3600	84	26	119	3600	3.85	120	(113,190)
24	20	5	6	2	No Solution Returned	No Solution Returned	3600	–	27	262	1199	0	263	(240,340)
25	30	5	6	2	No Solution Returned	No Solution Returned	3600	–	No Solution Returned	No Solution Returned	3600	–	341	(332,510)
26	3	10	6	2	44	30	3600	74	44	30	50	0	36	(30,51)
27	5	10	6	2	57	87	3600	73	50	89	26	0	91	(85,100)
28	10	10	6	2	418	138	3600	98	49	159	41	0	169	(126,190)
29	20	10	6	2	No Solution Returned	No Solution Returned	3600	–	52	310	3600	3.85	310	(280,396)
30	30	10	6	2	No Solution Returned	No Solution Returned	3600	–	50	450	116	0	556	(310,592)
31	3	3	3	4	19	47	37	0	19	47	2	0	47	(33,60)
32	5	3	3	4	22	64	101	0	22	64	2	0	66	(51,95)
33	10	3	3	4	28	130	3600	67	20	131	12	0	132	(106,200)
34	20	3	3	4	No Solution Returned	No Solution Returned	3600	–	24	251	3600	8.33	251	(228,399)
35	30	3	3	4	No Solution Returned	No Solution Returned	3600	–	23	406	3600	4.35	406	(313,585)
36	3	5	3	4	32	37	308	0	32	37	3	0	38	(32,55)
37	5	5	3	4	31	81	3600	25	31	81	13	0	89	(55,100)
38	10	5	3	4	73	157	3600	82	33	167	14	0	174	(110,192)
39	20	5	3	4	No Solution Returned	No Solution Returned	3600	–	38	220	3600	2.63	220	(200,381)
40	30	5	3	4	No Solution Returned	No Solution Returned	3600	–	41	413	140	0	417	(304,572)

(continued on next page)

Table 5 (continued)

Instance	No. jobs per echelon	No. stages/ operations per echelon	No. machines for each stage/ operation	No. factories per echelon	MILP		Solution Time (s)	Relative gap (%)	CP		Solution Time (s)	Relative gap (%)	ε	Range ε
					cmax	trCost			cmax	trCost				
41	3	10	3	4	67	43	3600	30	65	38	8	0	44	(33,50)
42	5	10	3	4	70	62	3600	57	70	61	10	0	65	(51,97)
43	10	10	3	4	291	115	3600	91	77	116	3600	2.6	116	(104,200)
44	20	10	3	4	No Solution Returned	No Solution Returned	3600	–	84	256	1628	0	258	(234,396)
45	30	10	3	4	No Solution Returned	No Solution Returned	3600	–	156	396	3600	55.13	396	(343,567)
46	3	3	6	4	9	43	26	0	9	43	2	0	48	(30,60)
47	5	3	6	4	18	77	3600	38	18	83	2	0	90	(55,94)
48	10	3	6	4	18	131	3600	83	15	126	29	0	132	(100,200)
49	20	3	6	4	54	275	3600	92	13	262	10	0	281	(200,399)
50	30	3	6	4	No Solution Returned	No Solution Returned	3600	–	14	461	20	0	552	(300,600)
51	3	5	6	4	23	38	3600	65	20	47	3	0	47	(30,57)
52	5	5	6	4	25	62	3600	76	24	70	5	0	79	(50,100)
53	10	5	6	4	41	147	3600	87	21	150	365	0	150	(100,200)
54	20	5	6	4	2257	220	3600	99	29	286	33	0	312	(200,400)
55	30	5	6	4	No Solution Returned	No Solution Returned	3600	–	23	391	63	0	463	(300,598)
56	3	10	6	4	47	34	3600	77	36	40	5	0	50	(30,59)
57	5	10	6	4	54	58	3600	89	51	66	163	0	66	(50,100)
58	10	10	6	4	197	152	3600	96	45	137	39	0	184	(110,190)
59	20	10	6	4	No Solution Returned	No Solution Returned	3600	–	49	310	123	0	391	(200,400)
60	30	10	6	4	No Solution Returned	No Solution Returned	3600	–	49	385	951	0	386	(300,570)
2nd Iteration														
61	3	3	3	2	23	48	11	0	23	48	2	0	48	(47,60)
62	5	3	3	2	26	77	72	0	26	77	2	0	77	(64,97)
63	10	3	3	2	37	117	3600	57	25	119	63	0	120	(102,184)
64	20	3	3	2	82	264	3600	82	27	265	19	0	265	(200,354)
65	30	3	3	2	1569	384	3600	99	33	458	3600	12.12	483	(372,528)
66	3	5	3	2	43	40	163	0	43	40	2	0	40	(36,55)
67	5	5	3	2	42	62	3600	40	39	61	11	0	63	(55,80)
68	10	5	3	2	96	132	3600	67	45	136	380	0	136	(118,176)
69	20	5	3	2	No Solution Returned	No Solution Returned	3600	–	42	302	30	0	302	(286,345)
70	30	5	3	2	3291	360	3600	99	48	408	3600	8.33	413	(335,510)
71	3	10	3	2	90	39	3600	30	90	39	12	0	40	(36,57)
72	5	10	3	2	114	57	3600	67	88	57	163	0	61	(57,82)
7	10	10	3	2	518	140	3600	96	75	150	12	0	152	(109,182)
74	20	10	3	2	No Solution Returned	No Solution Returned	3600	–	88	263	14	0	281	(212,315)
75	30	10	3	2	No Solution Returned	No Solution Returned	3600	–	85	492	3600	4.71	564	(380,590)
76	3	3	6	2	13	41	21	0	13	41	2	0	42	(39,48)
77	5	3	6	2	14	70	3600	7	14	67	2	0	74	(50,90)
78	10	3	6	2	23	132	3600	56	17	130	2	0	136	(110,170)
79	20	3	6	2	42	261	3600	88	15	255	7	0	290	(200,380)
80	30	3	6	2	2325	480	3600	99	18	544	52	0	555	(480,600)
81	3	5	6	2	22	36	3600	54	22	36	10	0	36	(33,51)
82	5	5	6	2	27	79	3600	50	24	78	2	0	80	(72,92)
83	10	5	6	2	34	149	3600	81	25	148	67	0	149	(130,200)

(continued on next page)

Table 5 (continued)

Instance	No. jobs per echelon	No. stages/ operations per echelon	No. machines for each stage/ operation	No. factories per echelon	MILP		Solution Time (s)	Relative gap (%)	CP		Solution Time (s)	Relative gap (%)	ε	Range ε
					cmax	trCost			cmax	trCost				
84	20	5	6	2	93	288	3600	89	26	275	16	0	350	(200,376)
85	30	5	6	2	3539	330	3600	99	29	449	99	0	512	300,600))
86	3	10	6	2	52	43	3600	83	44	44	4	0	46	(39,51)
87	5	10	6	2	59	74	3600	85	57	79	39	0	80	(65,85)
88	10	10	6	2	88	146	3600	90	50	146	417	0	146	(130,180)
89	20	10	6	2	No Solution Returned	No Solution Returned	3600	–	54	280	982	0	280	(261,320)
90	30	10	6	2	5987	392	3600	99	57	431	238	0	432	(392,506)
91	3	3	3	4	23	37	16	0	23	37	8	0	37	(35,59)
92	5	3	3	4	22	61	102	0	22	62	2	0	62	(51,99)
93	10	3	3	4	23	154	3600	43	22	151	10	0	154	(113,198)
94	20	3	3	4	74	236	3600	84	25	236	2177	0	236	(205,380)
95	30	3	3	4	70	439	3600	76	24	441	41	0	520	(303,591)
96	3	5	3	4	32	53	315	0	32	53	2	0	53	(30,60)
97	5	5	3	4	36	81	3600	28	34	74	3	0	82	(50,95)
98	10	5	3	4	67	133	3600	70	40	133	3600	2.5	133	(110,183)
99	20	5	3	4	173	264	3600	88	41	274	1430	0	274	(226,377)
100	30	5	3	4	1447	412	3600	98	37	437	181	0	461	(343,591)
101	3	10	3	4	85	41	3600	68	63	38	8	0	42	(30,54)
102	5	10	3	4	87	81	3600	34	86	79	7	0	91	(54,100)
103	10	10	3	4	109	146	3600	64	72	160	10	0	163	(112,190)
104	20	10	3	4	No Solution Returned	No Solution Returned	3600	–	84	315	28	0	351	(234,391)
105	30	10	3	4	No Solution Returned	No Solution Returned	3600	–	90	449	86	0	513	(337,584)
106	3	3	6	4	11	34	36	0	11	34	2	0	35	(30,60)
107	5	3	6	4	13	63	1346	0	13	63	12	0	63	(50,99)
108	10	3	6	4	19	127	3600	74	13	127	66	0	130	(100,200)
109	20	3	6	4	29	321	3600	93	15	312	11	0	333	(221,400)
110	30	3	6	4	197	455	3600	98	14	451	8	0	560	(300,600)
111	3	5	6	4	20	44	3600	44	20	42	2	0	55	(30,59)
112	5	5	6	4	26	68	3600	80	24	65	58	0	68	(55,100)
113	10	5	6	4	37	111	3600	82	26	111	3600	23.08	111	(102,200)
114	20	5	6	4	108	286	3600	96	22	298	35	0	310	(200,400)
115	30	5	6	4	3812	389	3600	99	23	449	51	0	450	(300,600)
116	3	10	6	4	41	37	3600	73	40	39	49	0	41	(30,60)
117	5	10	6	4	55	66	3600	90	42	72	18	0	76	(50,95)
118	10	10	6	4	183	120	3600	96	46	125	3600	0	125	(100,200)
119	20	10	6	4	No Solution Returned	No Solution Returned	3600	–	50	239	183	0	240	(200,397)
120	30	10	6	4	No Solution Returned	No Solution Returned	3600	–	49	440	3600	2.04	442	(300,600)
3rd Iteration														
121	3	3	3	2	26	37	14	0	26	37	2	0	37	(36,54)
122	5	3	3	2	26	68	497	0	26	68	2	0	68	50,90))
123	10	3	3	2	39	161	3600	64	25	161	2	0	164	(113,197)
124	20	3	3	2	163	326	3600	93	26	339	128	0	339	(282,399)
125	30	3	3	2	No Solution Returned	No Solution Returned	3600	–	37	381	3600	21.62	381	(312,545)
126	3	5	3	2	29	43	52	0	29	43	2	0	43	(33,60)
127	5	5	3	2	40	88	3600	22	35	88	5	0	91	(82,100)
128	10	5	3	2	62	147	3600	77	39	140	10	0	147	(116,169)

(continued on next page)

Table 5 (continued)

Instance	No. jobs per echelon	No. stages/ operations per echelon	No. machines for each stage/ operation	No. factories per echelon	MILP		Solution Time (s)	Relative gap (%)	CP		Solution Time (s)	Relative gap (%)	ε	Range ε
					cmax	trCost			cmax	trCost				
129	20	5	3	2	No Solution Returned	No Solution Returned	3600	–	43	257	30	0	300	(201,337)
130	30	5	3	2	No Solution Returned	No Solution Returned	3600	–	46	430	2247	0	442	(336,507)
131	3	10	3	2	84	49	3600	39	76	47	2	0	52	(41,56)
132	5	10	3	2	128	66	3600	71	83	66	97	0	70	(66,90)
133	10	10	3	2	254	146	3600	87	82	138	13	0	147	(116,177)
134	20	10	3	2	2452	298	3600	98	83	322	448	0	322	(298,371)
135	30	10	3	2	No Solution Returned	No Solution Returned	3600	–	86	391	3600	8.14	423	(300,494)
136	3	3	6	2	15	35	97	0	15	35	2	0	38	(33,45)
137	5	3	6	2	15	68	957	0	15	68	2	0	68	(60,95)
138	10	3	6	2	20	133	3600	55	16	127	2	0	140	(100,200)
139	20	3	6	2	85	301	3600	95	15	300	446	0	301	(260,398)
140	30	3	6	2	No Solution Returned	No Solution Returned	3600	–	18	472	28	0	568	(332,600)
141	3	5	6	2	23	36	3600	17	23	36	10	0	36	(30,42)
142	5	5	6	2	38	79	3600	70	26	79	2	0	81	(70,100)
143	10	5	6	2	61	118	3600	86	28	124	4	0	128	(110,150)
144	20	5	6	2	465	238	3600	98	25	240	31	0	240	(200,335)
145	30	5	6	2	4303	300	3600	99	29	405	31	0	411	(300,570)
146	3	10	6	2	51	41	3600	82	49	40	34	0	41	(33,50)
147	5	10	6	2	60	66	3600	83	49	66	51	0	68	(55,85)
148	10	10	6	2	101	145	3600	92	50	142	51	0	154	(100,180)
149	20	10	6	2	No Solution Returned	No Solution Returned	3600	–	No Solution Returned	No Solution Returned	3600	–	212	(205,360)
150	30	10	6	2	No Solution Returned	No Solution Returned	3600	–	59	467	70	0	506	(392,540)
151	3	3	3	4	24	45	42	0	24	45	2	0	46	(33,60)
152	5	3	3	4	25	67	63	0	25	67	5	0	67	(60,98)
153	10	3	3	4	30	150	3600	65	23	151	3	0	163	(102,200)
154	20	3	3	4	No Solution Returned	No Solution Returned	3600	–	27	291	11	0	294	(205,373)
155	30	3	3	4	No Solution Returned	No Solution Returned	3600	–	No Solution Returned	No Solution Returned	3600	–	396	(366,594)
156	3	5	3	4	33	45	608	0	33	45	2	0	45	(30,59)
157	5	5	3	4	33	76	3600	51	30	76	8	0	76	(50,99)
158	10	5	3	4	66	138	3600	59	40	135	6	0	154	(100,183)
159	20	5	3	4	ns	ns	3600	–	41	304	27	0	306	(224,397)
160	30	5	3	4	1979	406	3600	99	35	468	55	0	510	(300,565)
161	3	10	3	4	74	34	3600	43	69	35	5	0	42	(31,54)
162	5	10	3	4	90	70	3600	53	80	71	31	0	84	(51,92)
163	10	10	3	4	142	135	3600	66	83	132	31	0	135	(112,167)
164	20	10	3	4	1913	314	3600	98	80	290	55	0	315	(239,366)
165	30	10	3	4	No Solution Returned	No Solution Returned	3600	–	83	405	386	0	419	(325,586)
166	3	3	6	4	12	35	167	0	12	35	11	0	38	(30,60)
167	5	3	6	4	12	65	3600	16	11	62	2	0	66	(50,100)
168	10	3	6	4	20	124	3600	85	12	125	45	0	125	(100,200)
169	20	3	6	4	1464	200	3600	99	14	267	80	0	267	(200,398)
170	30	3	6	4	3190	360	3600	99	16	412	10	0	443	(300,596)
171	3	5	6	4	19	37	3600	52	19	37	3	0	39	(30,57)
172	5	5	6	4	23	79	3600	73	23	80	4	0	82	(50,100)

(continued on next page)

Table 5 (continued)

Instance	No. jobs per echelon	No. stages/ operations per echelon	No. machines for each stage/ operation	No. factories per echelon	MILP		Relative gap (%)	CP cmax	trCost	Solution Time (s)	Relative gap (%)	Solution Time (s)	Relative gap (%)	ε	Range ε
					cmax	trCost									
173	10	5	6	4	38	131	84	21	133	5	0	186	0	186	(100,200)
174	20	5	6	4	2149	240	99	24	280	14	0	305	0	305	(200,398)
175	30	5	6	4	3417	360	99	26	418	58	0	542	0	542	(330,600)
176	3	10	6	4	41	43	80	41	42	5	0	45	0	45	(30,57)
177	5	10	6	4	62	63	86	49	63	3600	8.16	63	8.16	63	(52,100)
178	10	10	6	4	648	170	98	46	149	75	0	172	0	172	(100,190)
179	20	10	6	4	No Solution Returned	No Solution Returned	–	49	249	155	0	258	0	258	(200,400)
180	30	10	6	4	No Solution Returned	No Solution Returned	–	51	406	1899	0	410	0	410	(300,570)

model.

In part (b) of Fig. 6, the average run time of the next 15 instances for three iterations can be seen. These instances have six machines per stage/operation and two factories per echelon. The CP model obtained the optimal solution for all cases within the time limit. On the other hand, the MILP model achieved an optimal solution only for the first two cases which have three and five jobs, respectively.

The numerical instances in part (c) of Fig. 6 have three machines per stage/operation and four factories for each echelon. Like the previous 15 instances, the CP model obtained the optimal solution for all the instances in this category within 3600 s, but the MILP model achieved the optimal solution for only three instances and with a significant difference in terms of the run time of the CP model. Also, in part (d) of Fig. 5, the instances have six machines for each stage/operation and four factories per level. The CP model also provided the optimal solution for all cases of this category in an extremely short time. Meanwhile, the MILP model acquired the optimal solution for only a few numbers of instances, just like the previous categories. In most cases, it can be concluded that the CP model has obtained the optimal solution for all the generated random instances in a much shorter time than the MILP model, which indicates the superiority of this approach for solving the complex problem considered in this research. Table 4 illustrates a summary of the results of using the MILP and CP approaches to solve the instances.

In the last two lines of Table 4, the total and average solution status of the instances in 3600 s is calculated. As it is known, in a total of three iterations, the number of instances that are solved with the MILP model in a time limit of 3600 s with a zero solution gap and provided the optimal solution is equal to 27. The number of cases that are not solved at all is equal to 37, and the number of instances that are solved with a significant solution gap is equal to 116. Meanwhile, this classification for CP model instances is 157, 4, and 19, respectively. This shows the outstanding ability of the CP method to solve scheduling problems with high dimensions in a reasonable time. In Fig. 7, the trend of solution gap changes for the instances that do not provide an optimal solution in 3600 s by two software can be seen:

As it is clear from Fig. 7, the first three points marked on each curve represent the average percentage of the solution gap for the instances solved in each iteration by two models. The blue dot on each graph also shows the average percentage of the solution gap for all instances, which reveals the prominent difference between these two numbers. In fact, a difference of about 63 units proves the absolutely appropriate efficiency of the CP method for solving complex distributed production scheduling problems.

8. Conclusion and future studies

In this research, a bi-objective MILP model is proposed for the distributed production scheduling problem considering a two-echelon network configuration where the first and second echelon's factories had a hybrid flow shop and flexible job shop configuration, respectively. The objective functions of the presented model sought to minimize the maximum completion time of jobs and transportation costs between the selected factories of two echelons, respectively, and the epsilon constraint method is used to deal with this bi-objective model.

Since the distributed scheduling problems in the real environment are classified as NP-Hard problems even in the simplest case, the proposed problem is certainly not an exception to this reality (Bargaoui et al., 2017); (Wu et al., 2017). On the other hand, paying attention to the run time is one of the important factors in scheduling problems. In other words, due to the daily need for solving this kind of model in each production unit, the desired model should be solved in a short time and a suitable solution should be obtained. Therefore, concerning this issue, in addition to the MILP, a model based on the concepts of the CP paradigm is presented. This model can solve problems with larger sizes in a reasonable time, which showed the satisfactory performance of this method in the previous sections. In fact, to solve each of the two models

Table 6

The average solution time of solving the two models for 180 numerical instances.

Instance	No. jobs per echelon	No. stages/operations per echelon	No. machines for each stage/operation	No. factories per echelon	Solution time for MILP model in 1 st iteration (s)	Solution time for MILP model in 2 nd iteration (s)	Solution time for MILP model in 3 rd iteration (s)	Average Solution time for MILP model (s)	Solution time for CP model in 1 st iteration (s)	Solution time for CP model in 2 nd iteration (s)	Solution time for CP model in 3 rd iteration (s)	Average Solution time for CP model (s)
1	3	3	3	2	11	11	14	12	2	2	2	2
2	5	3	3	2	729	72	497	432.67	3	2	2	2.33
3	10	3	3	2	3600	3600	3600	3600	3	63	2	22.67
4	20	3	3	2	3600	3600	3600	3600	36	19	128	61
5	30	3	3	2	3600	3600	3600	3600	3600	3600	3600	3600
6	3	5	3	2	73	163	52	96	3	2	2	2.33
7	5	5	3	2	3600	3600	3600	3600	37	11	5	17.67
8	10	5	3	2	3600	3600	3600	3600	11	380	10	133.67
9	20	5	3	2	3600	3600	3600	3600	24	30	30	28
10	30	5	3	2	3600	3600	3600	3600	3600	3600	2247	3149
11	3	10	3	2	3600	3600	3600	3600	3	12	2	5.67
12	5	10	3	2	3600	3600	3600	3600	16	163	97	92
13	10	10	3	2	3600	3600	3600	3600	19	12	13	14.67
14	20	10	3	2	3600	3600	3600	3600	47	14	448	169.67
15	30	10	3	2	3600	3600	3600	3600	3600	3600	3600	3600
16	3	3	6	2	47	21	97	55	2	2	2	2
17	5	3	6	2	155	3600	957	1570.67	12	2	2	5.33
18	10	3	6	2	3600	3600	3600	3600	3	2	2	2.33
19	20	3	6	2	3600	3600	3600	3600	3600	7	446	1351
20	30	3	6	2	3600	3600	3600	3600	215	52	28	98.33
21	3	5	6	2	3600	3600	3600	3600	16	10	10	12
22	5	5	6	2	3600	3600	3600	3600	4	2	2	2.67
23	10	5	6	2	3600	3600	3600	3600	3600	67	4	1223.67
24	20	5	6	2	3600	3600	3600	3600	1199	16	31	415.33
25	30	5	6	2	3600	3600	3600	3600	3600	99	31	1243.33
26	3	10	6	2	3600	3600	3600	3600	50	4	34	26
27	5	10	6	2	3600	3600	3600	3600	26	39	51	38.67
28	10	10	6	2	3600	3600	3600	3600	41	417	51	169.67
29	20	10	6	2	3600	3600	3600	3600	3600	982	3600	2727.33
30	30	10	6	2	3600	3600	3600	3600	116	238	70	141.33
31	3	3	3	4	37	16	42	31.67	2	8	2	4
32	5	3	3	4	101	102	63	88.67	2	2	5	3
33	10	3	3	4	3600	3600	3600	3600	12	10	3	8.33
34	20	3	3	4	3600	3600	3600	3600	3600	2177	11	1929.33
35	30	3	3	4	3600	3600	3600	3600	3600	41	3600	2413.67
36	3	5	3	4	308	315	608	410.33	3	2	2	2.33
37	5	5	3	4	3600	3600	3600	3600	13	3	8	8
38	10	5	3	4	3600	3600	3600	3600	14	3600	6	1206.67
39	20	5	3	4	3600	3600	3600	3600	3600	1430	27	1685.67
40	30	5	3	4	3600	3600	3600	3600	140	181	55	125.33
41	3	10	3	4	3600	3600	3600	3600	8	8	5	7
42	5	10	3	4	3600	3600	3600	3600	10	7	31	16
43	10	10	3	4	3600	3600	3600	3600	3600	10	31	1213.67
44	20	10	3	4	3600	3600	3600	3600	1628	28	55	570.33
45	30	10	3	4	3600	3600	3600	3600	3600	86	386	1357.33
46	3	3	6	4	26	36	167	76.33	2	2	11	5
47	5	3	6	4	3600	1346	3600	2848.67	2	12	2	5.33
48	10	3	6	4	3600	3600	3600	3600	29	66	45	31.67
49	20	3	6	4	3600	3600	3600	3600	10	11	80	33.7
50	30	3	6	4	3600	3600	3600	3600	20	8	10	12.67
51	3	5	6	4	3600	3600	3600	3600	3	2	3	2.67
52	5	5	6	4	3600	3600	3600	3600	5	58	4	22.33
53	10	5	6	4	3600	3600	3600	3600	365	3600	5	1323.33
54	20	5	6	4	3600	3600	3600	3600	33	35	14	27.33
55	30	5	6	4	3600	3600	3600	3600	63	51	58	57.33
56	3	10	6	4	3600	3600	3600	3600	5	49	5	19.67
57	5	10	6	4	3600	3600	3600	3600	163	18	3600	1260.33
58	10	10	6	4	3600	3600	3600	3600	39	3600	75	1238
59	20	10	6	4	3600	3600	3600	3600	123	183	155	153.67
60	30	10	6	4	3600	3600	3600	3600	951	3600	1899	2150

Table 7

The relative difference of the first objective function resulting from solving the instances for three iterations (the detailed table).

Group	Instance	No. jobs per echelon	No. stages/ operations per echelon	No. machines for each stage/ operation	No. factories per echelon	The relative difference of the first objective function in 1st iteration	The relative difference of the first objective function in 2nd iteration	The relative difference of the first objective function in 3rd iteration
1	1	3	3	3	2	0	0	0
	2	5	3	3	2	0.0416	0	0
	3	10	3	3	2	-0.518	-0.48	-0.56
	4	20	3	3	2	-3.926	-2.037	-5.269
	5	30	3	3	2	-11.675	-46.545	-
2	6	3	5	3	2	0	0	0
	7	5	5	3	2	0	-0.077	-0.143
	8	10	5	3	2	-1.225	-1.133	-0.589
	9	20	5	3	2	-26.309	-	-
	10	30	5	3	2	-	-67.562	-
3	11	3	10	3	2	-0.011	0	0.105
	12	5	10	3	2	-0.294	-0.295	0.542
	13	10	10	3	2	-2.215	-5.906	-2.097
	14	20	10	3	2	-34.18	-	-28.542
	15	30	10	3	2	-	-	-
4	16	3	3	6	2	0	0	0
	17	5	3	6	2	0	0	0
	18	10	3	6	2	-0.294	-0.353	-0.25
	19	20	3	6	2	-1.75	-1.8	-4.66
	20	30	3	6	2	-	-128.166	-
5	21	3	5	6	2	0	0	0
	22	5	5	6	2	0	-0.125	-0.461
	23	10	5	6	2	-0.654	-0.36	-1.178
	24	20	5	6	2	-	-2.577	-17.6
	25	30	5	6	2	-	-121.034	-147.379
6	26	3	10	6	2	0	-0.182	-0.041
	27	5	10	6	2	-0.14	-0.035	-0.224
	28	10	10	6	2	-7.53	-0.76	-1.02
	29	20	10	6	2	-	-	-
	30	30	10	6	2	-	-104.035	-
7	31	3	3	3	4	0	0	0
	32	5	3	3	4	0	0	0
	33	10	3	3	4	-0.4	-0.045	-0.304
	34	20	3	3	4	-	-1.96	-
	35	30	3	3	4	-	-1.916	-
8	36	3	5	3	4	0	0	0
	37	5	5	3	4	0	-0.059	-0.1
	38	10	5	3	4	-1.21	-0.675	-0.65
	39	20	5	3	4	-	-3.219	-
	40	30	5	3	4	-	-38.108	-55.543
9	41	3	10	3	4	-0.03	-0.349	-0.072
	42	5	10	3	4	0	-0.011	-0.125
	43	10	10	3	4	-2.779	-0.514	-0.711
	44	20	10	3	4	-	-	-22.912
	45	30	10	3	4	-	-	-
10	46	3	3	6	4	0	0	0
	47	5	3	6	4	0	0	-0.09
	48	10	3	6	4	-0.2	-0.461	-0.66
	49	20	3	6	4	-3.154	-0.933	-103.571
	50	30	3	6	4	-	-13.071	-198.375
11	51	3	5	6	4	-0.15	0	0
	52	5	5	6	4	-0.041	-0.083	0
	53	10	5	6	4	-0.952	-0.423	-0.809
	54	20	5	6	4	-76.827	-3.909	-88.541
	55	30	5	6	4	-	-164.739	-130.423
12	56	3	10	6	4	-0.305	-0.025	0
	57	5	10	6	4	-0.058	-0.309	-0.265
	58	10	10	6	4	-3.377	-2.978	-13.087
	59	20	10	6	4	-	-	-
	60	30	10	6	4	-	-	-

presented in this research, a total of 180 random numerical instances were made for small, medium, and large sizes, and the MILP and CP models are solved by GAMS and CPLEX software respectively.

The MILP model could optimally solve some instances that have a maximum of five jobs. In other cases, it either provided a solution with a non-negligible gap compared to the optimal solution, or no integer solution was obtained at all within the time limit. On the other hand, the CP model had the noticeable ability to solve a very large number of instances, optimally and in much less time in comparison with the MILP model, even for the instances with 30 jobs. Only in a few cases, the CP

model was unable to obtain a solution or generated solutions with a small gap from the optimal value. Taking into account these results, we can strongly suggest the CP method for complex scheduling problems, especially in the field of distributed scheduling.

Considering the activities carried out in this study and the apparent need to continue relevant activities in this field, the following are suggested future works for the continuation of research by researchers interested in this field:

- Using the data of a real case study and adjusting the parameters of the presented problem accordingly.
- Using heuristic and meta-heuristic solution methods to solve the mathematical model and comparing the run time and the quality of the obtained solution with the CP approach results.
- Considering uncertainty for some important parameters of the problem, including processing times or transportation costs, and trying to solve it using conventional approaches to face uncertainty.
- Development of the proposed model for other traditional shop configurations in production scheduling problems.
- Considering other objective functions such as earliness/tardiness penalty, number of late jobs, and even objective functions related to sustainable development.
- Implementation of Industry 4.0 concepts to convert the presented model into a smart distributed scheduling model.
- Comparison of the performance of the proposed models with state-of-the-art algorithms and models for scheduling problems.

CRedit authorship contribution statement

Ghazal Ziadlou: Conceptualization, Data curation, Formal analysis, Methodology, Resources, Software, Validation, Writing – original draft, Writing – review & editing. **Saeed Emami:** . **Ebrahim Asadi-Gangraj:** Conceptualization, Methodology, Validation, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Appendix

See Tables 5-7

References

- Bargaoui, H., Driss, O. B., & Ghédira, K. (2017). A novel chemical reaction optimization for the distributed permutation flowshop scheduling problem with makespan criterion. *Computers & Industrial Engineering*, 111, 239–250.
- Bonomi, F., & Kumar, A. (1990). Adaptive optimal load balancing in a nonhomogeneous multiserver system with a central job scheduler. *IEEE Transactions on Computers*, 39(10), 1232–1250.
- Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations research*, 41(3), 157–183.
- Chan, F. T., Chung, S. H., & Chan, P. (2005). An adaptive genetic algorithm with dominated genes for distributed scheduling problems. *Expert Systems with Applications*, 29(2), 364–371.
- CSP, C.-E. (2020). Constraint Satisfaction Problems.
- Deb, K. (2014). Multi-objective optimization. In *Search methodologies* (pp. 403–449). Springer.
- El Khayat, G., Langevin, A., & Riopel, D. (2006). Integrated production and material handling scheduling using mathematical programming and constraint programming. *European Journal of Operational Research*, 175(3), 1818–1832.
- Fathollahi-Fard, A. M., Woodward, L., & Akhrif, O. (2021). Sustainable distributed permutation flow-shop scheduling model based on a triple bottom line concept. *Journal of Industrial Information Integration*, 24, Article 100233.
- Fernandez-Viagas, V., & Framinan, J. M. (2020). Design of a testbed for hybrid flow shop scheduling with identical machines. *Computers & Industrial Engineering*, 141, Article 106288.
- Gedik, R., Rainwater, C., Nachtmann, H., & Pohl, E. A. (2016). Analysis of a parallel machine scheduling problem with sequence dependent setup times and job availability intervals. *European Journal of Operational Research*, 251(2), 640–650.
- Gholami, H., & Sun, H. (2023). Toward automated algorithm configuration for distributed hybrid flow shop scheduling with multiprocessor tasks. *Knowledge-Based Systems*, 264, Article 110309.
- Glover, F., & Woolsey, E. (1974). Converting the 0–1 polynomial programming problem to a 0–1 linear program. *Operations research*, 22(1), 180–182.
- Gökgür, B., Hnich, B., & Özpeynirci, S. (2018). Parallel machine scheduling with tool loading: A constraint programming approach. *International Journal of Production Research*, 56(16), 5541–5557.
- Gonzalez-Neira, E. M., Ferone, D., Hatami, S., & Juan, A. A. (2017). A biased-randomized simheuristic for the distributed assembly permutation flowshop problem with stochastic processing times. *Simulation Modelling Practice and Theory*, 79, 23–36.
- Guo, Z., Yang, C., Wang, W., & Yang, J. (2015). Harmony search-based multi-objective optimization model for multi-site order planning with multiple uncertainties and learning effects. *Computers & Industrial Engineering*, 83, 74–90.
- H'Mida, F., & Lopez, P. (2013). Multi-site scheduling under production and transportation constraints. *International Journal of Computer Integrated Manufacturing*, 26(3), 252–266.
- Haimes, Y. (1971). On a bicriterion formulation of the problems of integrated system identification and system optimization. *IEEE transactions on systems, man, and cybernetics*, 1(3), 296–297.
- Hou, Y., Fu, Y., Gao, K., Zhang, H., & Sadollah, A. (2022). Modelling and optimization of integrated distributed flow shop scheduling and distribution problems with time windows. *Expert Systems with Applications*, 187, Article 115827.
- Huang, Y.-Y., Pan, Q.-K., Huang, J.-P., Suganthan, P. N., & Gao, L. (2021). An improved iterated greedy algorithm for the distributed assembly permutation flowshop scheduling problem. *Computers & Industrial Engineering*, 152, Article 107021.
- Jiang, Z., Yuan, S., Ma, J., & Wang, Q. (2022). The evolution of production scheduling from Industry 3.0 through Industry 4.0. *International Journal of Production Research*, 60(11), 3534–3554.
- Kanet, J.J., Ahire, S.L., & Gorman, M.F. (2004). Constraint programming for scheduling. Karabulut, K., Kizilay, D., Tasgetiren, M. F., Gao, L., & Kandiller, L. (2022). An evolution strategy approach for the distributed blocking flowshop scheduling problem. *Computers & Industrial Engineering*, 163, Article 107832.
- Karp, R. M. (1972). Reducibility among combinatorial problems. In *Complexity of computer computations* (pp. 85–103). Springer.
- Kelbel, J., & Hanzálek, Z. (2011). Solving production scheduling with earliness/tardiness penalties by constraint programming. *Journal of Intelligent Manufacturing*, 22(4), 553–562.
- Laborie, P., Rogerie, J., Shaw, P., & Vilím, P. (2018). IBM ILOG CP optimizer for scheduling. *Constraints*, 23(2), 210–250.
- Lei, D., & Liu, M. (2020). An artificial bee colony with division for distributed unrelated parallel machine scheduling with preventive maintenance. *Computers & Industrial Engineering*, 141, Article 106320.
- Lei, D., Yuan, Y., Cai, J., & Bai, D. (2020). An imperialist competitive algorithm with memory for distributed unrelated parallel machines scheduling. *International Journal of Production Research*, 58(2), 597–614.
- Lohmer, J., & Lasch, R. (2021). Production planning and scheduling in multi-factory production networks: A systematic literature review. *International Journal of Production Research*, 59(7), 2028–2054.
- Marandi, F., & Fatemi Ghomi, S. (2019). Integrated multi-factory production and distribution scheduling applying vehicle routing approach. *International Journal of Production Research*, 57(3), 722–748.
- Marandi, F., & Ghomi, S. F. (2019). Network configuration multi-factory scheduling with batch delivery: A learning-oriented simulated annealing approach. *Computers & Industrial Engineering*, 132, 293–310.
- Mavrotas, G. (2009). Effective implementation of the ϵ -constraint method in multi-objective mathematical programming problems. *Applied mathematics and computation*, 213(2), 455–465.
- Meng, L., Zhang, C., Ren, Y., Zhang, B., & Lv, C. (2020). Mixed-integer linear programming and constraint programming formulations for solving distributed flexible job shop scheduling problem. *Computers & Industrial Engineering*, 142, Article 106347.
- Naderi, B., & Ruiz, R. (2010). The distributed permutation flowshop scheduling problem. *Computers & Operations Research*, 37(4), 754–768.
- Novas, J. M. (2019). Production scheduling and lot streaming at flexible job-shops environments using constraint programming. *Computers & Industrial Engineering*, 136, 252–264.
- Olhager, J., & Feldmann, A. (2018). Distribution of manufacturing strategy decision-making in multi-plant networks. *International Journal of Production Research*, 56(1–2), 692–708.
- Pinedo, M. L. (2012). *Scheduling*. (Vol. 29): Springer.
- Rossi, F., Van Beek, P., & Walsh, T. (2006). *Handbook of constraint programming*. Elsevier.
- Sauer, J. (1998). A Multi-Site Scheduling System. In: AAAI's Special Interest Group in Manufacturing Workshop on Artificial Intelligence and Manufacturing: State of the Art and State of Practice (pp. 161–168).
- Shahmoradi, H., Ketabi, S., & Esmaelian, M. (2017). University Course Timetabling using Constraint Programming. *Journal of Production and Operations Management*, 8(1), 119–138.
- Shao, W., Shao, Z., & Pi, D. (2023). Modelling and optimization of distributed heterogeneous hybrid flow shop lot-streaming scheduling problem. *Expert Systems with Applications*, 214, Article 119151.
- Smith, W.E. (1955). Various optimizers for single-stage production. In: CALIFORNIA UNIV LOS ANGELES NUMERICAL ANALYSIS RESEARCH.
- Thoney, K. A., Hodgson, T. J., King, R. E., Taner, M. R., & Wilson, A. D. (2002). Satisfying due-dates in large multi-factory supply chains. *IIE Transactions*, 34(9), 803–811.
- Toptal, A., & Sabuncuoglu, I. (2010). Distributed scheduling: A review of concepts and applications. *International Journal of Production Research*, 48(18), 5235–5262.
- Wang, Z., Parhi, S. S., Rangaiah, G. P., & Jana, A. K. (2020). Analysis of weighting and selection methods for pareto-optimal solutions of multiobjective optimization in chemical engineering applications. *Industrial & Engineering Chemistry Research*, 59(33), 14850–14867.

- Wang, Z., & Rangaiah, G. P. (2017). Application and analysis of methods for selecting an optimal solution from the Pareto-optimal front obtained by multiobjective optimization. *Industrial & Engineering Chemistry Research*, 56(2), 560–574.
- Wang, P. S., Yang, T., & Yu, L.-C. (2018). Lean-pull strategy for order scheduling problem in a multi-site semiconductor crystal ingot-pulling manufacturing company. *Computers & Industrial Engineering*, 125, 545–562.
- Williams, J. F. (1981). Heuristic techniques for simultaneous scheduling of production and distribution in multi-echelon structures: Theory and empirical comparisons. *Management Science*, 27(3), 336–352.
- Wu, M.-C., Lin, C.-S., Lin, C.-H., & Chen, C.-F. (2017). Effects of different chromosome representations in developing genetic algorithms to solve DFJS scheduling problems. *Computers & Operations Research*, 80, 101–112.
- Yang, S., & Xu, Z. (2021). The distributed assembly permutation flowshop scheduling problem with flexible assembly and batch delivery. *International Journal of Production Research*, 59(13), 4053–4071.